JENKINS-TASK-01

1.Install Jenkins and run Jenkins on port number 8081.

Jenkins Installation and Configuration on Port 8081

Objective: To install Jenkins on an Amazon Linux 2023 EC2 instance, configure Jenkins to run on **port 8081**, start the Jenkins service, and access the Jenkins dashboard through a web browser.

Prerequisites

Before installing Jenkins, make sure you have:

- AWS account
- EC2 instance
- Amazon Linux 2023
- SSH access to the EC2 instance
- Internet connectivity from the EC2 instance
- Security Group access
- Java 21
- Port **8081** allowed in the Security Group

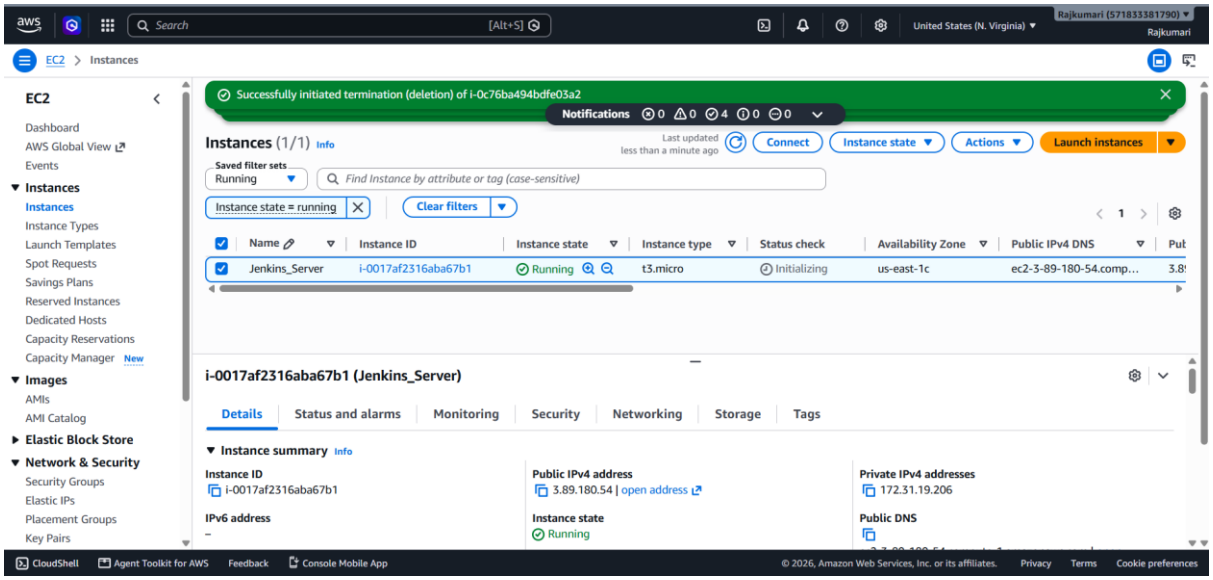
Step 1 – Connect to the EC2 Instance

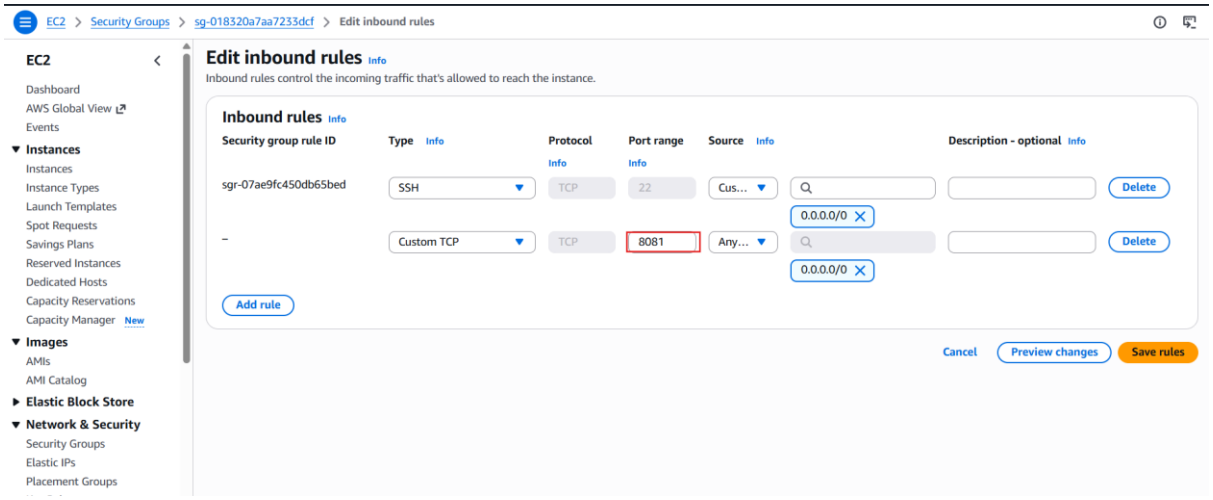
Open your AWS Console.

Go to: **EC2 → Instances → Select your EC2 Instance → Connect**

Select: **EC2 Instance Connect**

or connect using SSH.





Step 2 – Update the System Packages

Run: sudo dnf update -y

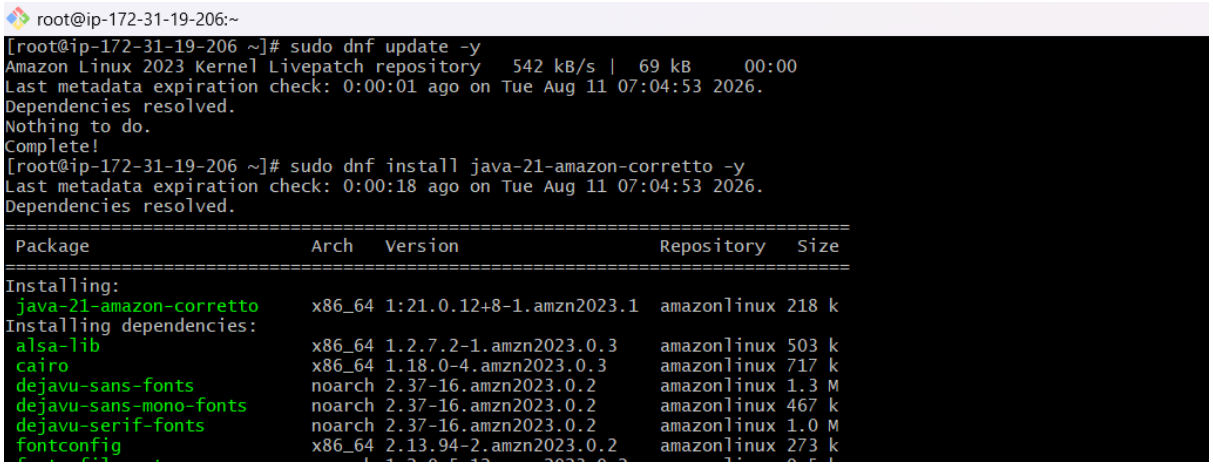
Step 3 – Install Java 21

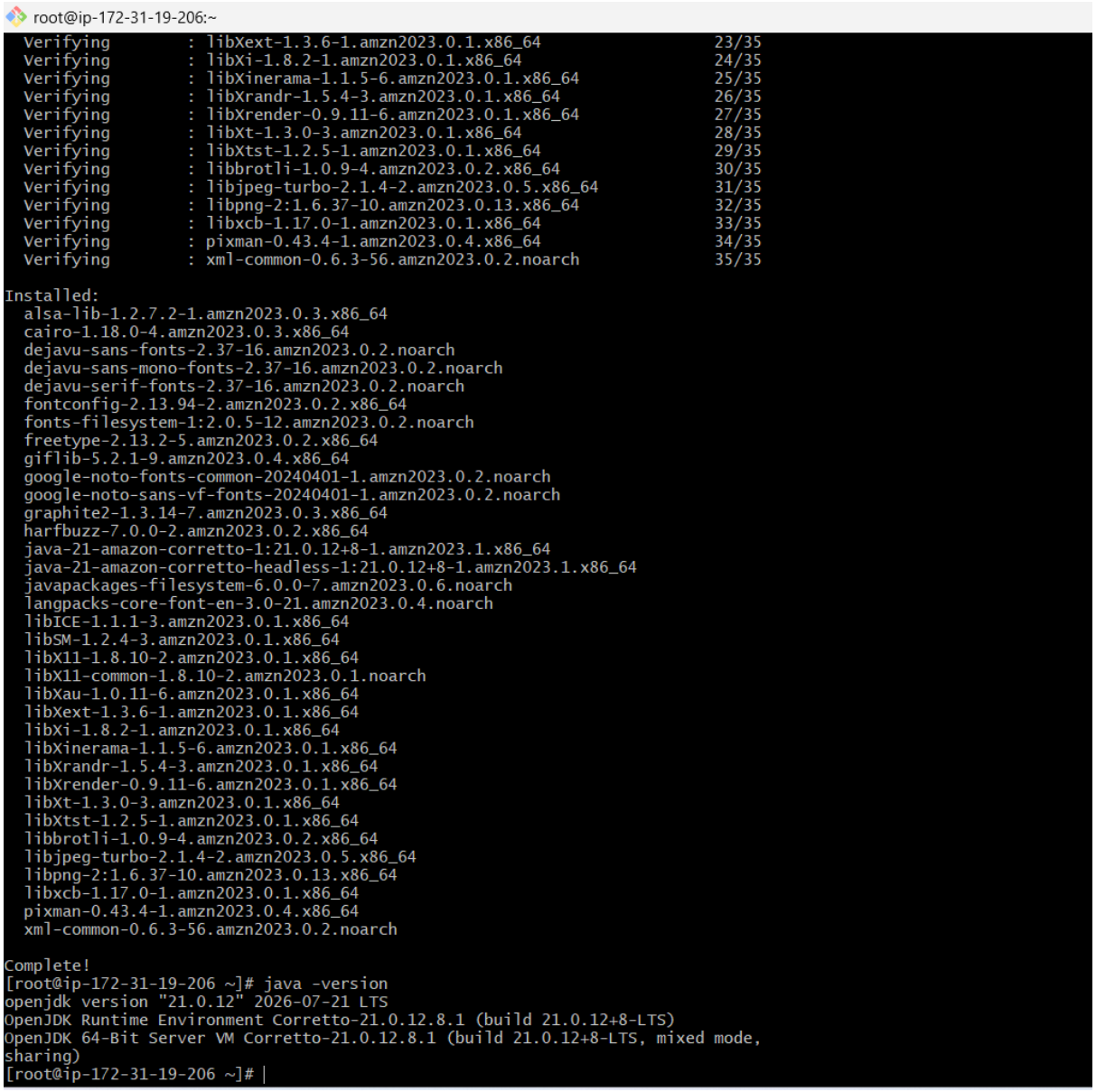
Jenkins requires Java to run. Current Jenkins LTS releases support Java 21, making it a suitable choice for this assignment.

Run: sudo dnf install java-21-amazon-corretto -y

Step 4 – Verify Java Installation

Run: java -version





Step 5 – Add the Jenkins Repository

Download the Jenkins LTS repository:

sudo wget -O /etc/yum.repos.d/jenkins.repo \

https://pkg.jenkins.io/rpm-stable/jenkins.repo

The official Jenkins documentation provides the RPM repository method for Red Hat-based Linux distributions.

Step 6 – Import Jenkins Repository Key

Run: sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key

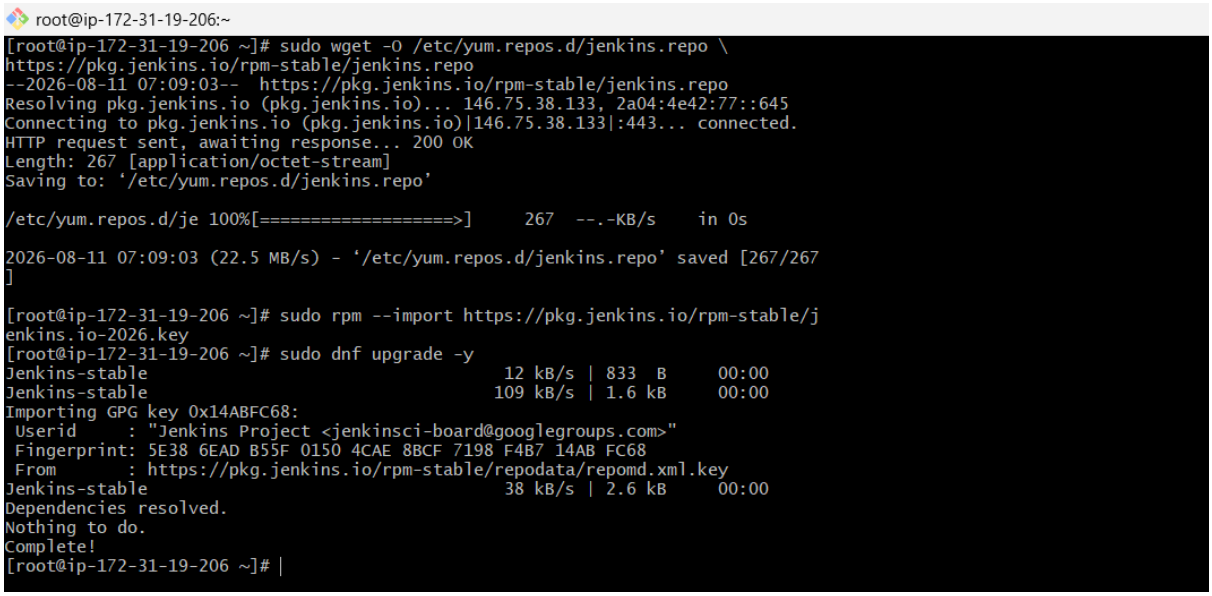
This imports the Jenkins repository signing key so that packages can be verified.

The current Jenkins AWS installation tutorial provides this repository-key step.

Step 7 – Refresh Package Information

Run: sudo dnf upgrade -y

This refreshes and upgrades packages using the configured repositories.



Step 8 – Install Jenkins

Run: sudo dnf install jenkins -y

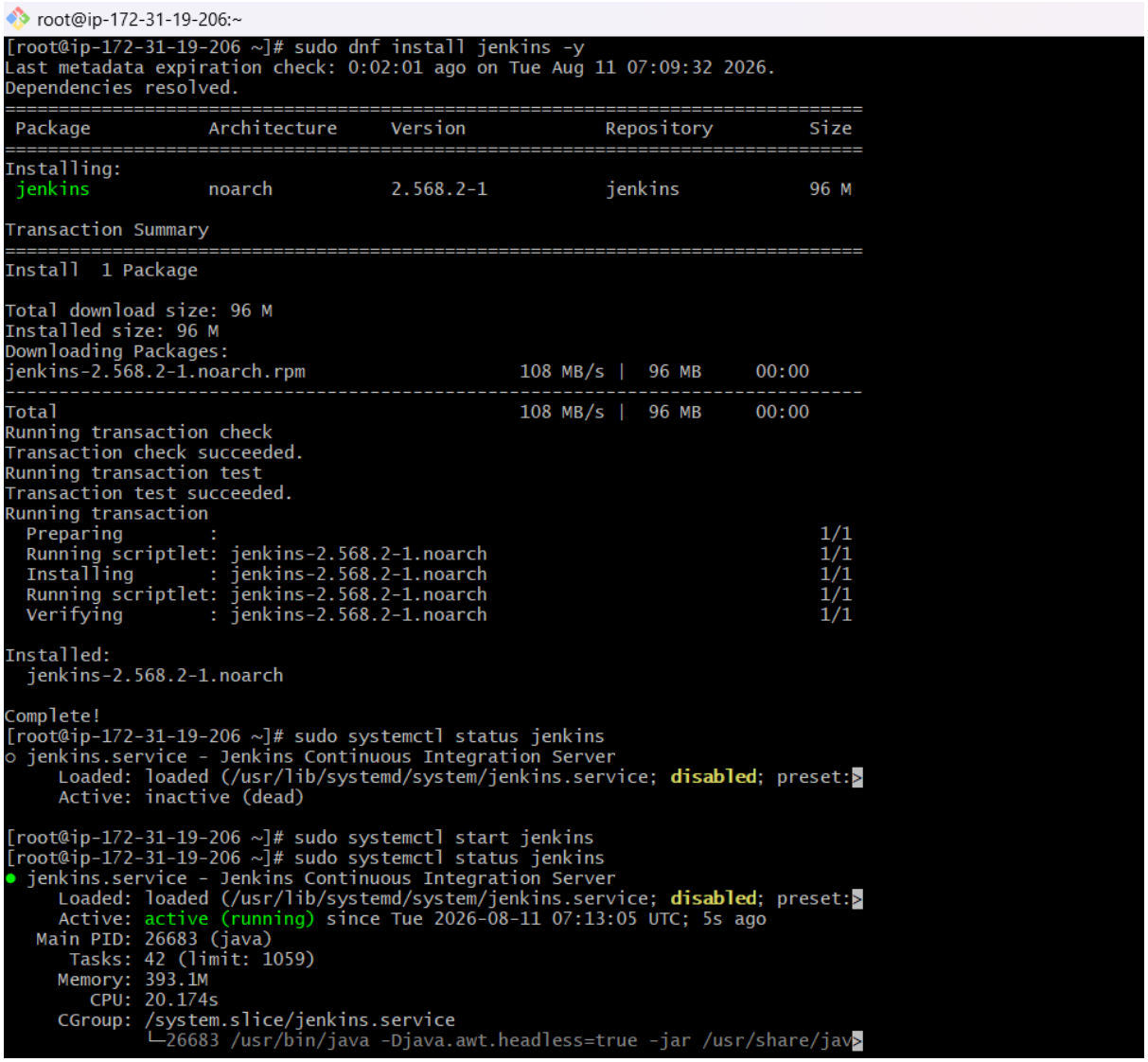
Wait until the installation finishes.

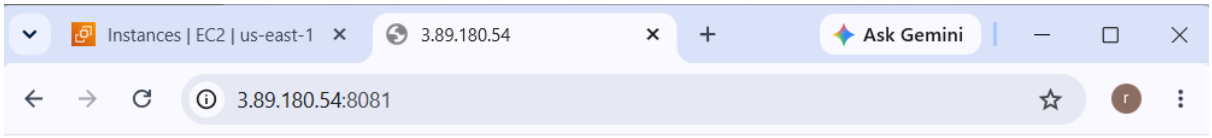
Jenkins will be installed as a system service.

The Jenkins package creates a dedicated jenkins user and configures Jenkins as a service managed by system

Step 9 – Verify Jenkins Installation

Check the Jenkins service: sudo systemctl status Jenkins





This site can't be reached

3.89.180.54 took too long to respond.

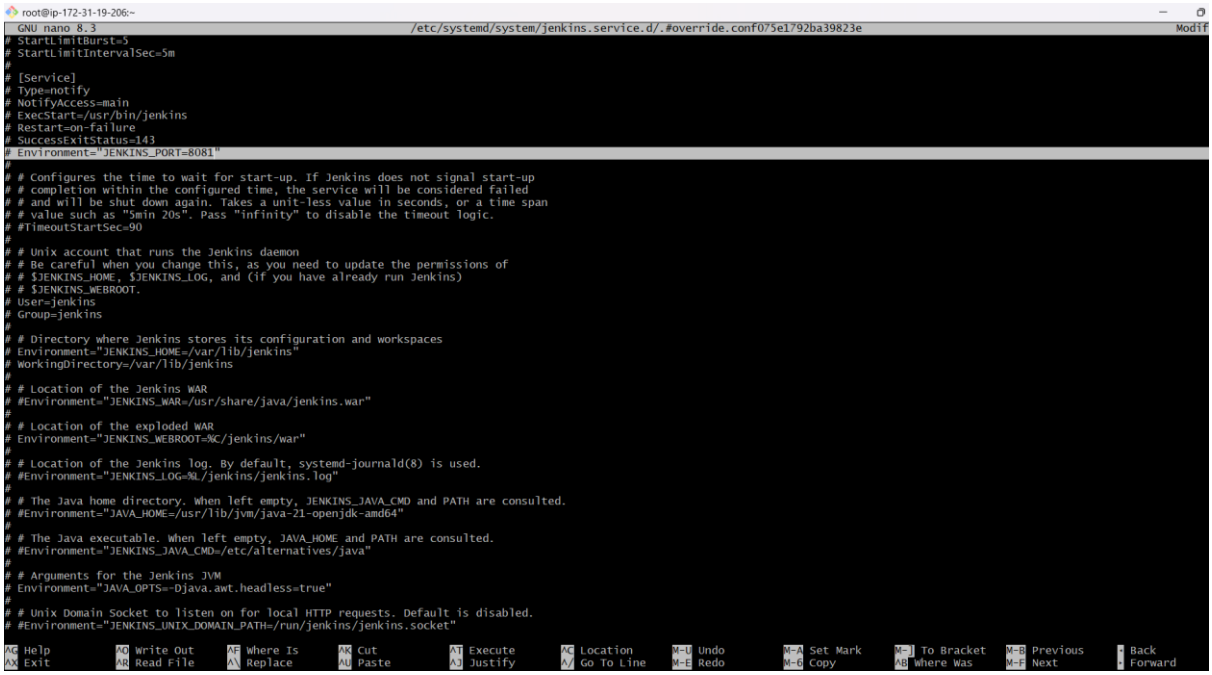
Step 10 – Configure Jenkins to Run on Port 8081

This is the **important step for your assignment**.

By default Jenkins uses: 8080

You need to change it to: 8081

Run: sudo systemctl edit Jenkins



Step 11 – Reload systemd

Run: sudo systemctl daemon-reload

Step 12 – Enable Jenkins at System Startup

Run: sudo systemctl enable Jenkins

Step 13 – Start Jenkins

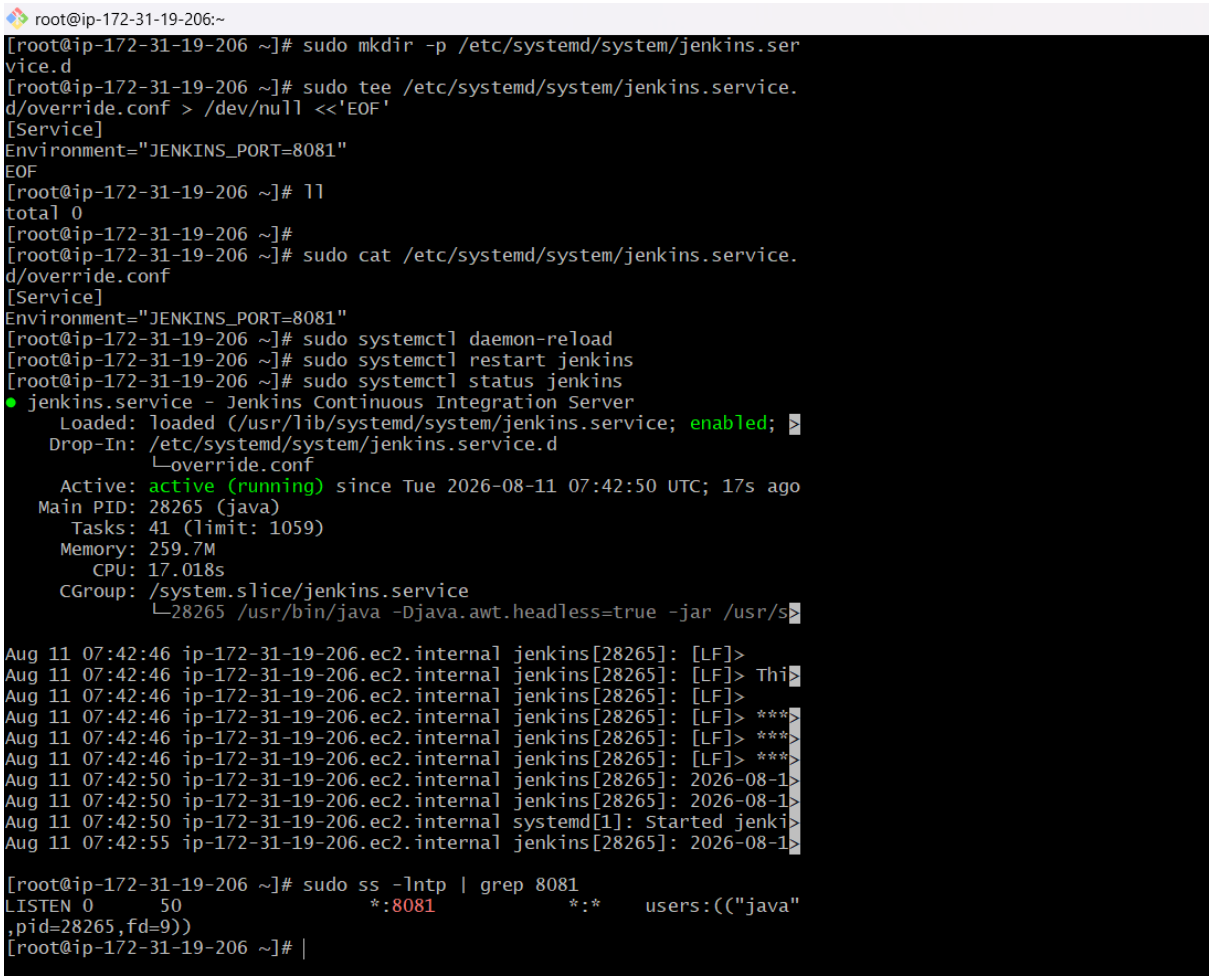
Run: sudo systemctl start Jenkins

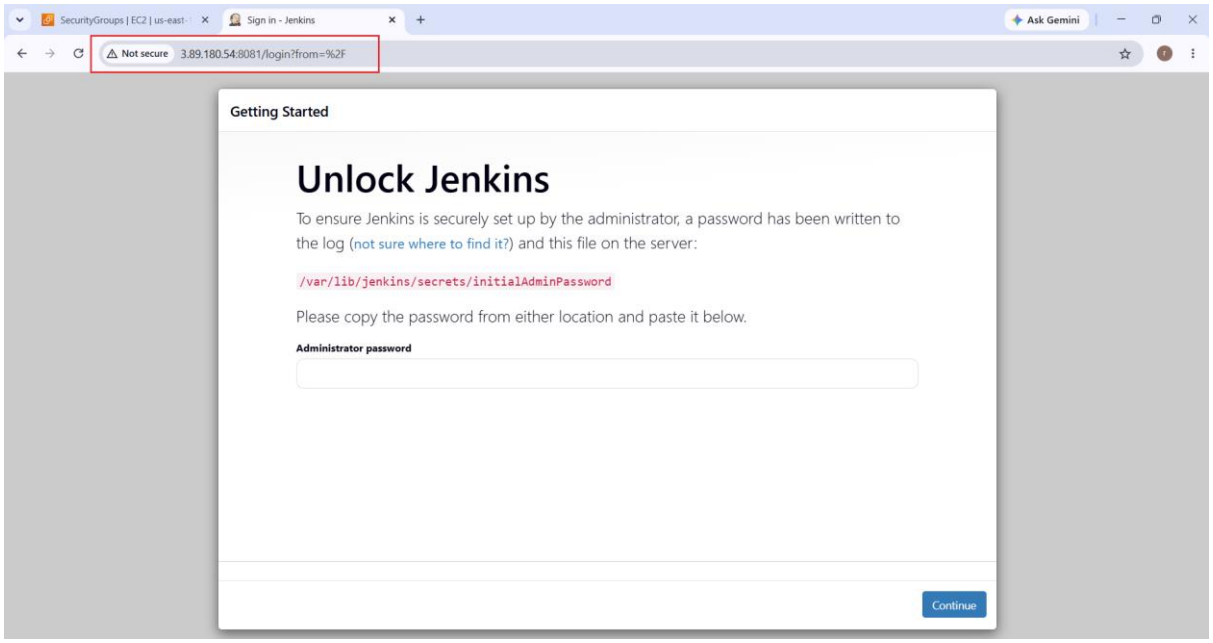
Run: sudo systemctl status Jenkins

Step 14 – Verify Jenkins Port

Now verify that Jenkins is actually listening on **8081**.

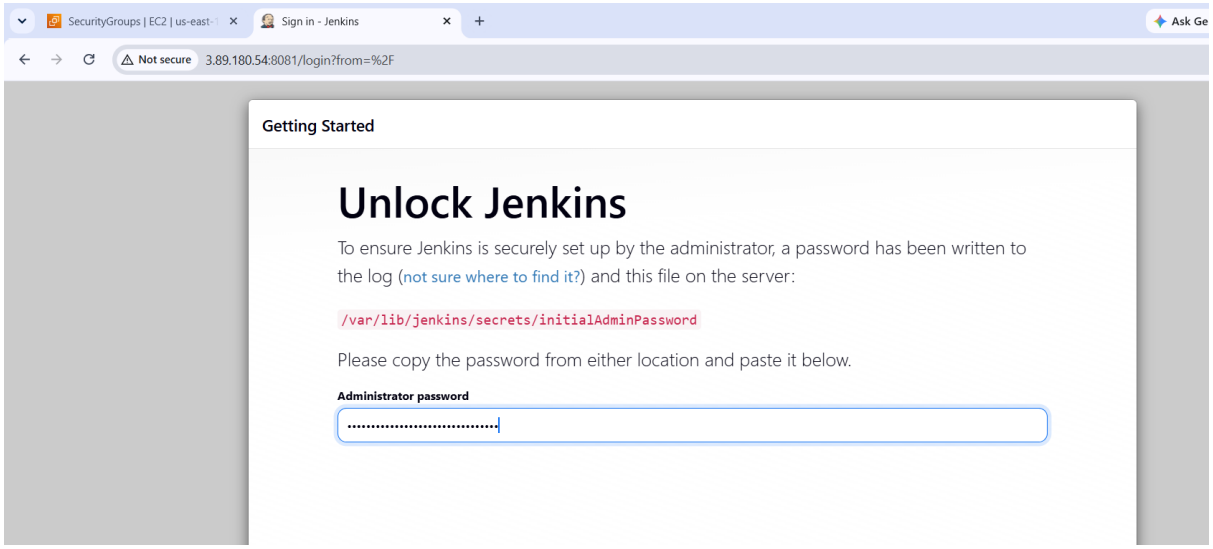
Run: sudo ss -lntp | grep 8081

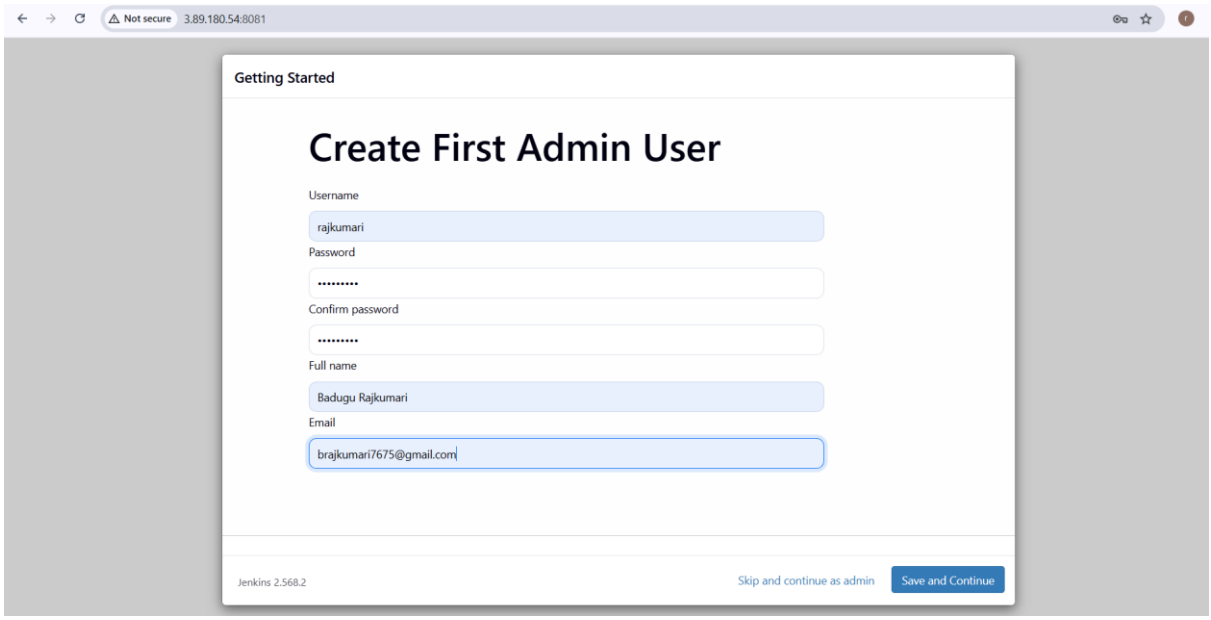


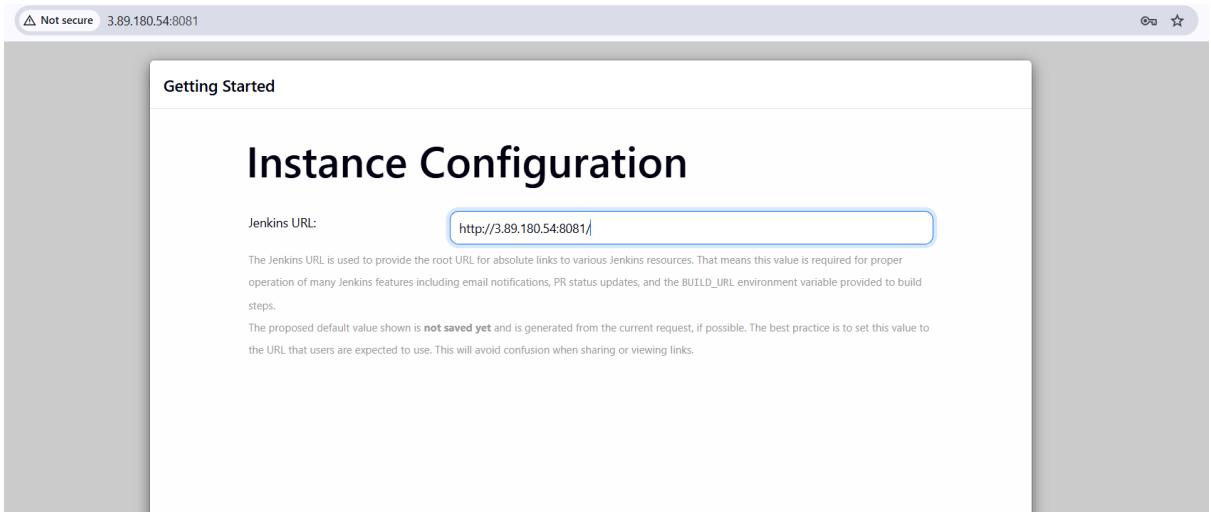


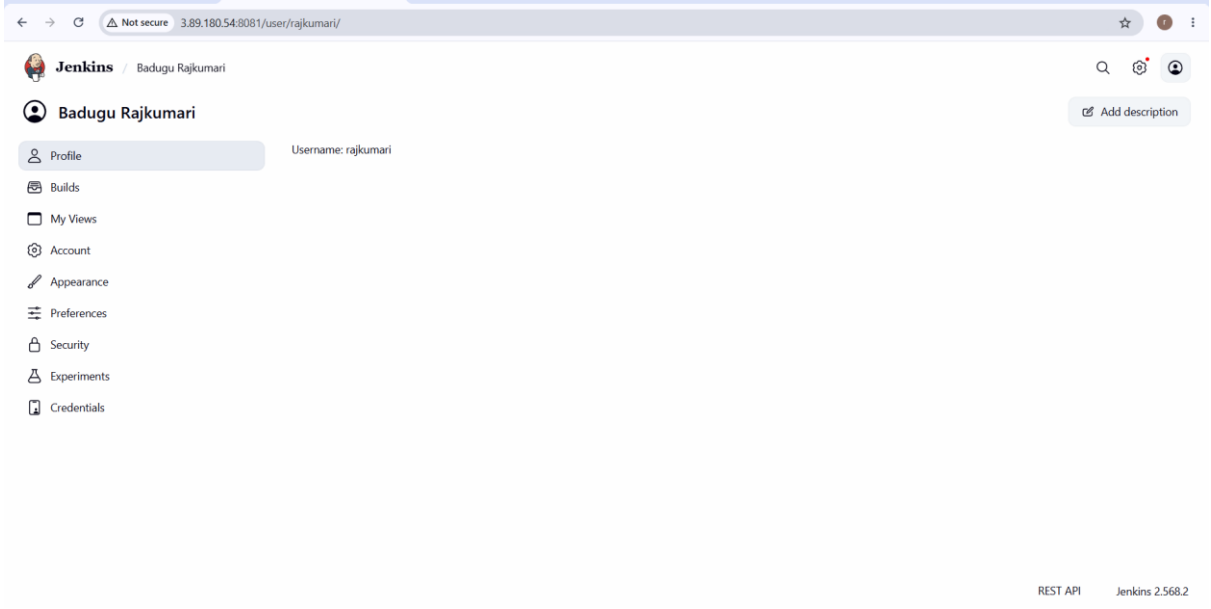
Step 15 – Retrieve Initial Administrator Password

On the EC2 instance, run: sudo cat /var/lib/jenkins/secrets/initialAdminPassword




Not secure 3.89.180.54:8081
Getting Started
Create First Admin User
Username
rajkumari
Password
Confirm password
Full name
Badugu Rajkumari
Email
brajkumari7675@gmail.com
Jenkins 2.568.2
Skip and continue as admin
Save and Continue


Not secure 3.89.180.54:8081
Getting Started
Instance Configuration
Jenkins URL:
http://3.89.180.54:8081/
The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.
The proposed default value shown is not saved yet and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.


Not secure 3.89.180.54:8081/user/rajkumari/
Jenkins / Badugu Rajkumari
Badugu Rajkumari
Add description
Profile
Builds
My Views
Account
Appearance
Preferences
Security
Experiments
Credentials
Username: rajkumari
REST API
Jenkins 2.568.2

2. Secure Jenkins server

Objective: To secure the Jenkins server by configuring authentication, authorization, CSRF protection, Security Group restrictions, secure administrator access, and other recommended Jenkins security settings.

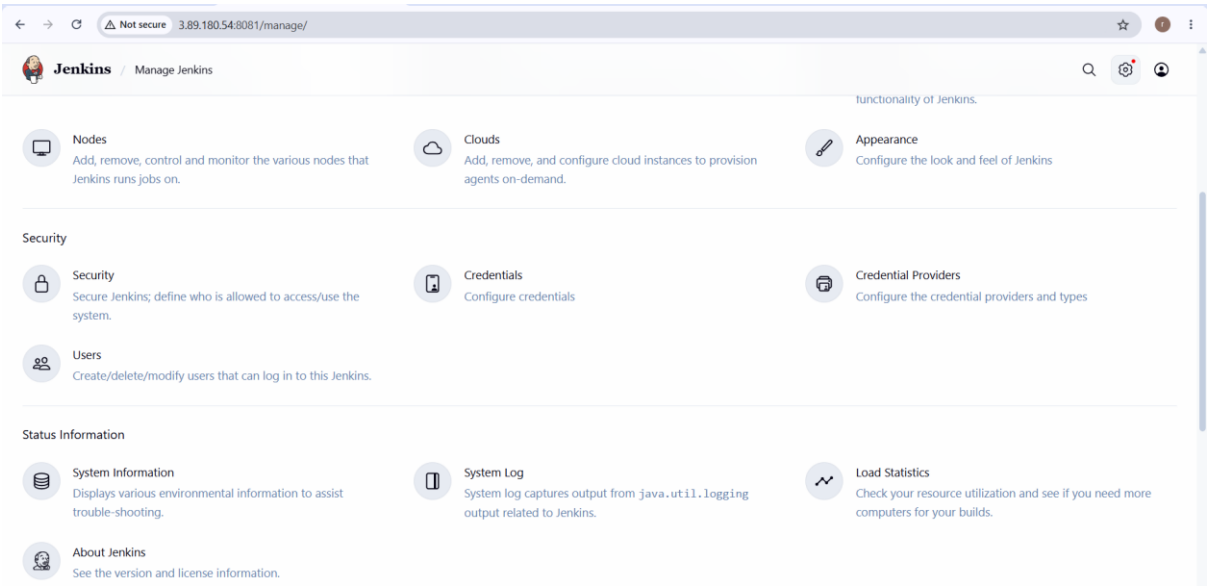
Step 1 – Verify Jenkins Security Is Enabled

Open Jenkins: http://<EC2-PUBLIC-IP>:8081

Log in using your Jenkins administrator account.

Go to: **Manage Jenkins → Security**

Depending on your Jenkins version, the exact menu may appear as **Manage Jenkins → Security** or under the security configuration area.



Step 2 – Configure Authentication

Authentication answers:

Who are you?

Jenkins should not allow anonymous users to administer the server.

In the Security configuration, verify that Jenkins uses:

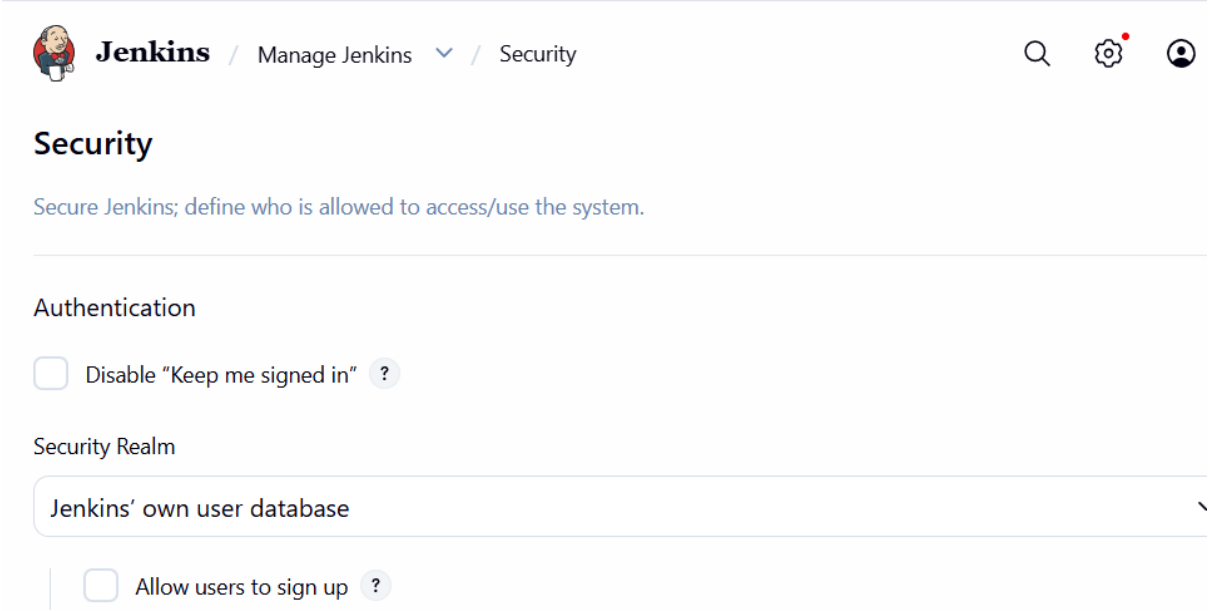
Security Realm → Jenkins' own user database

For a basic lab environment, this is sufficient.

Make sure:

Allow users to sign up

is **disabled** unless you specifically need users to create their own accounts



Step 3 – Configure Authorization

Authorization answers:

What are you allowed to do?

Under **Authorization**, use a role-based or matrix-based authorization strategy where appropriate.

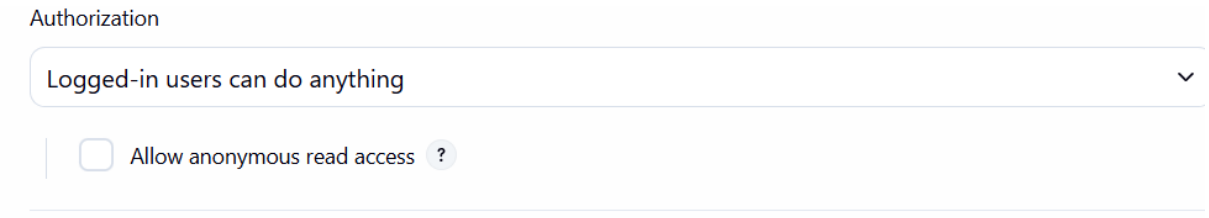
For a simple assignment, you can use:

Logged-in users can do anything

This means:

- Anonymous users → No administrative access
- Authenticated users → Jenkins access according to the selected strategy

For a production environment, use **least privilege** rather than allowing every authenticated user to do everything.



Verify CSRF Protection

CSRF means:

Cross-Site Request Forgery

It is an attack where a malicious website attempts to make an authenticated user perform an unwanted action.

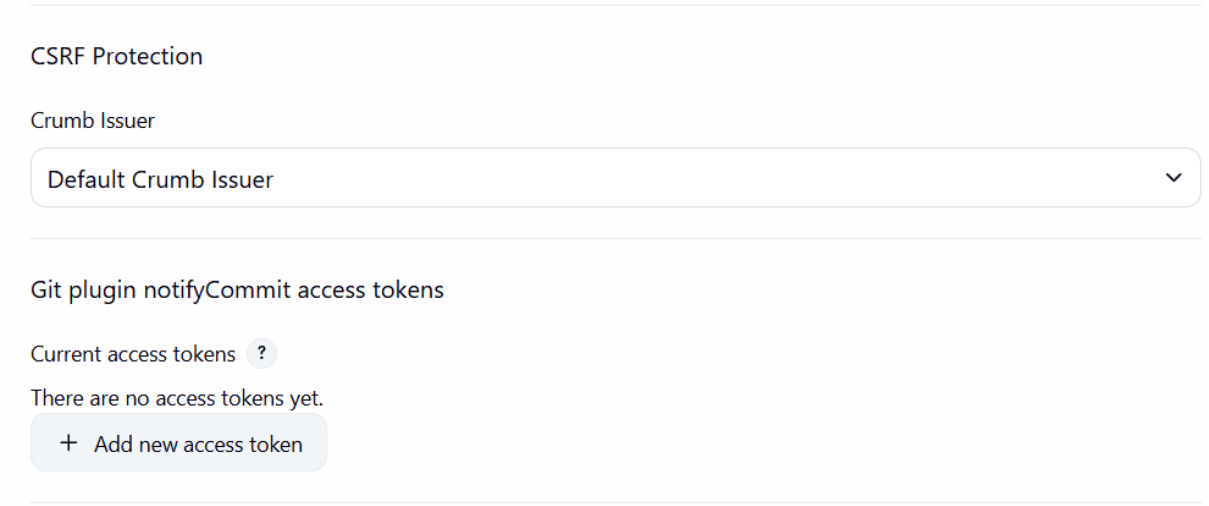
Jenkins provides CSRF protection using a **crumb** mechanism.

In the Jenkins security configuration, verify that:

CSRF Protection

is enabled.

Do **not** disable CSRF protection simply to make an integration work. Instead, configure the integration correctly.

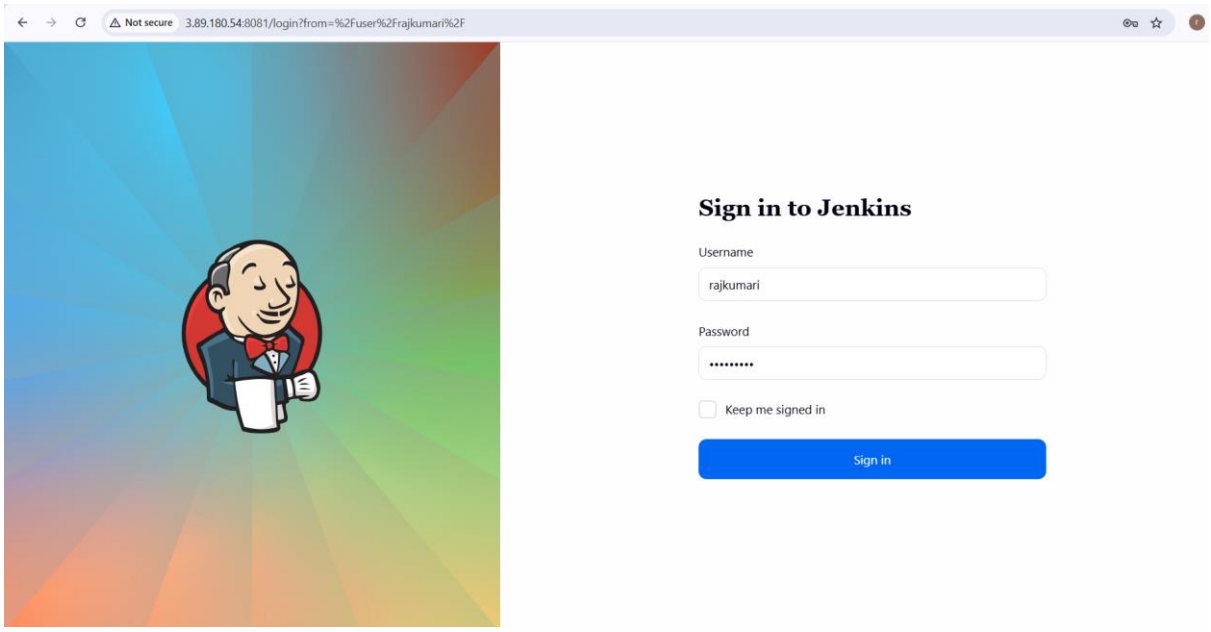


Step 5 – Secure the Jenkins Administrator Account

Go to:

Jenkins → Your Username → Configure

Use a strong password.



Step 6 – Secure the AWS Security Group

This is one of the most important security controls for your EC2 Jenkins server.

Go to:

AWS Console → EC2 → Instances → Your Jenkins Instance → Security → Security Groups

Select:

Inbound rules → Edit inbound rules

Your SSH rule should preferably be:

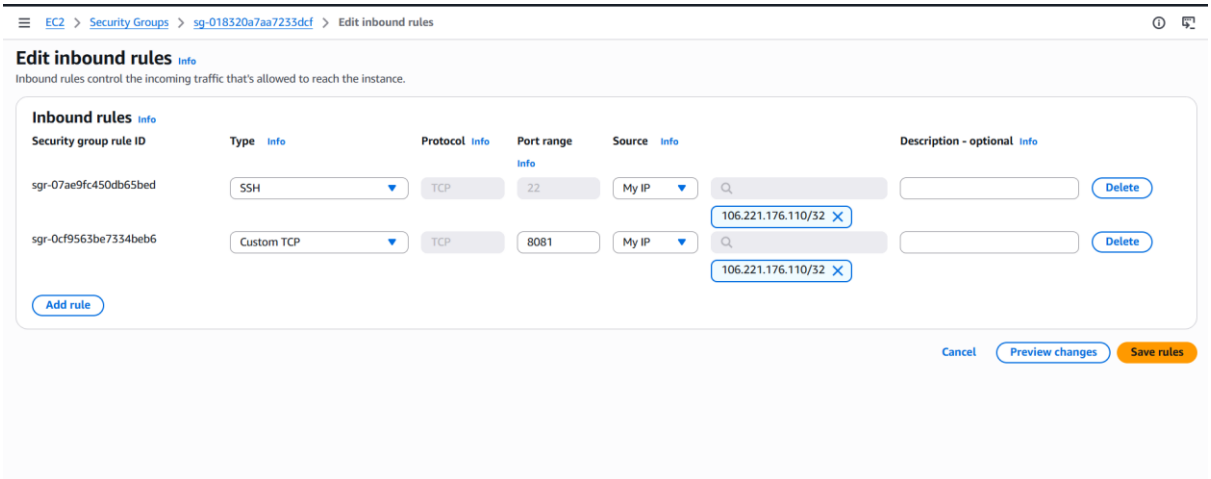
Type	Port	Source
SSH	22	My IP
Custom TCP	8081	My IP

Avoid this for Jenkins:

8081 → 0.0.0.0/0

because it makes Jenkins accessible from the entire Internet.

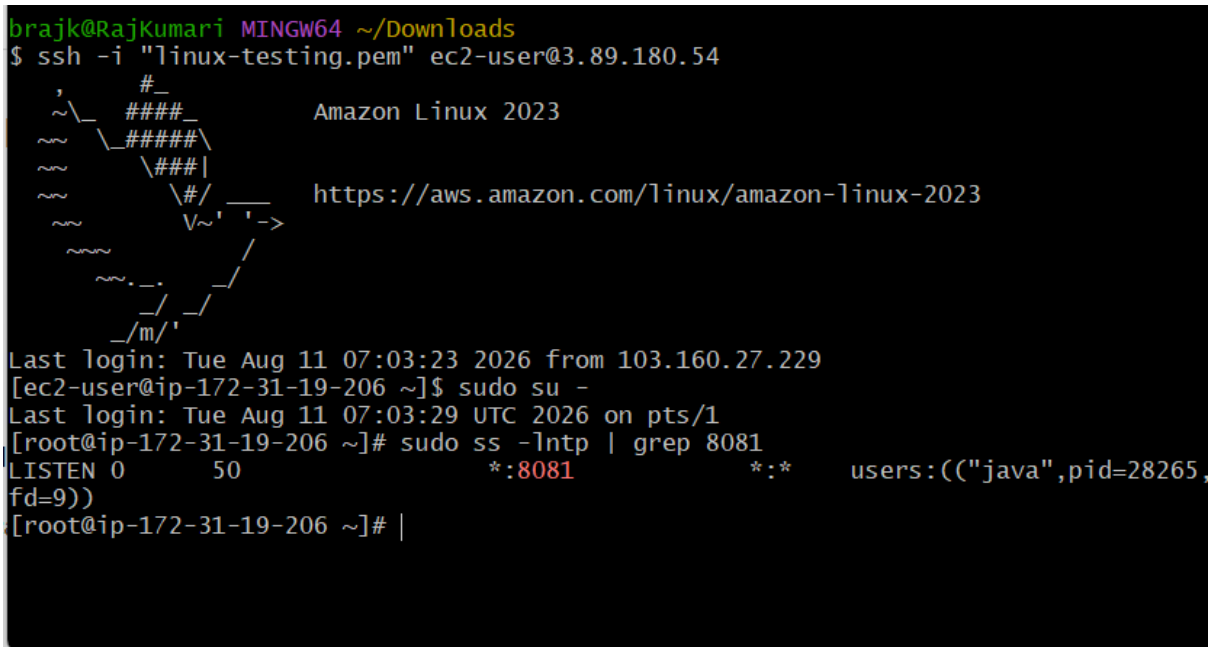
For a learning environment, **My IP** is much safer.



Step 8 – Verify Jenkins Port

On your EC2 instance:

sudo ss -lntp | grep 8081



Step 9 – Disable Unnecessary Anonymous Access

Test your Jenkins URL in a private/incognito browser window:

http://<EC2-PUBLIC-IP>:8081

Jenkins should require authentication.

You should see the login page rather than being able to access the Jenkins dashboard anonymously.

This confirms that anonymous users cannot access administrative functionality.

Step 10 – Keep Jenkins Updated

Regularly check: **Manage Jenkins → Plugins**

and: **Manage Jenkins → System**

Keep Jenkins and installed plugins updated.

This is important because Jenkins plugins can introduce security vulnerabilities if they are outdated.

Only install plugins that you actually need.

Step 11 – Install Only Required Plugins

Go to: **Manage Jenkins → Plugins**

Review installed plugins.

Remove unnecessary plugins when appropriate.

A good security principle is: **Install the minimum number of plugins required for your CI/CD pipeline.**

Every additional plugin increases the software surface that needs to be maintained and secured.

3. Create users called DevOps, Testing in Jenkins with Limited access.

Step 1: Open Jenkins in your web browser and log in using an account with **Administrator** privileges.

Step 2: Open Manage Jenkins

From the Jenkins dashboard, click:

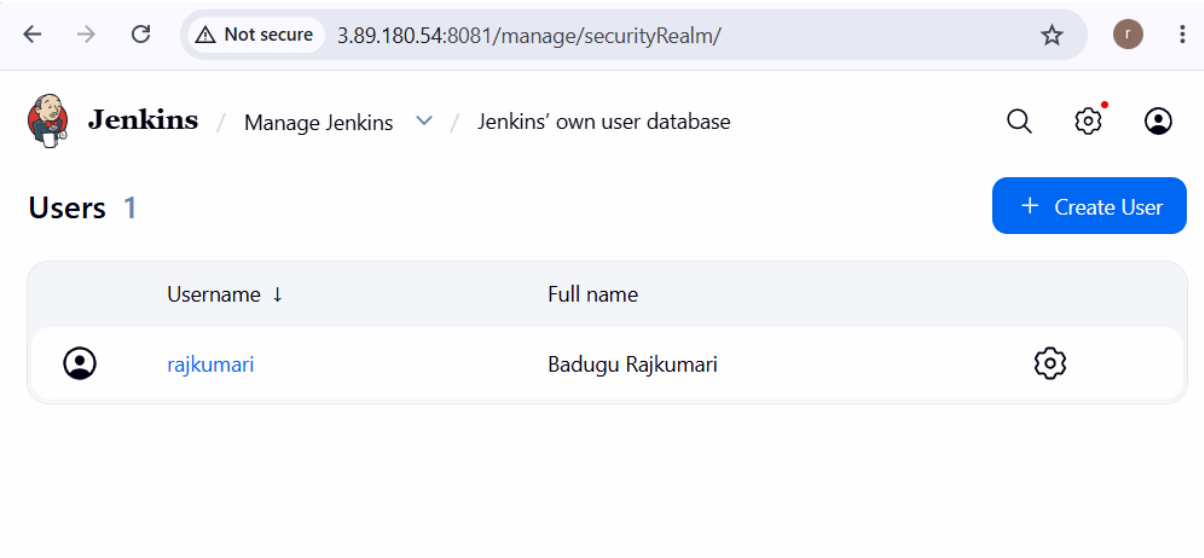
Manage Jenkins

Step 3: Open Users

Under the **Security** or **System Configuration** section, select:

Users

If **Users** is not directly visible, go to **Manage Jenkins → Users**.



Step 4: Create the DevOps User

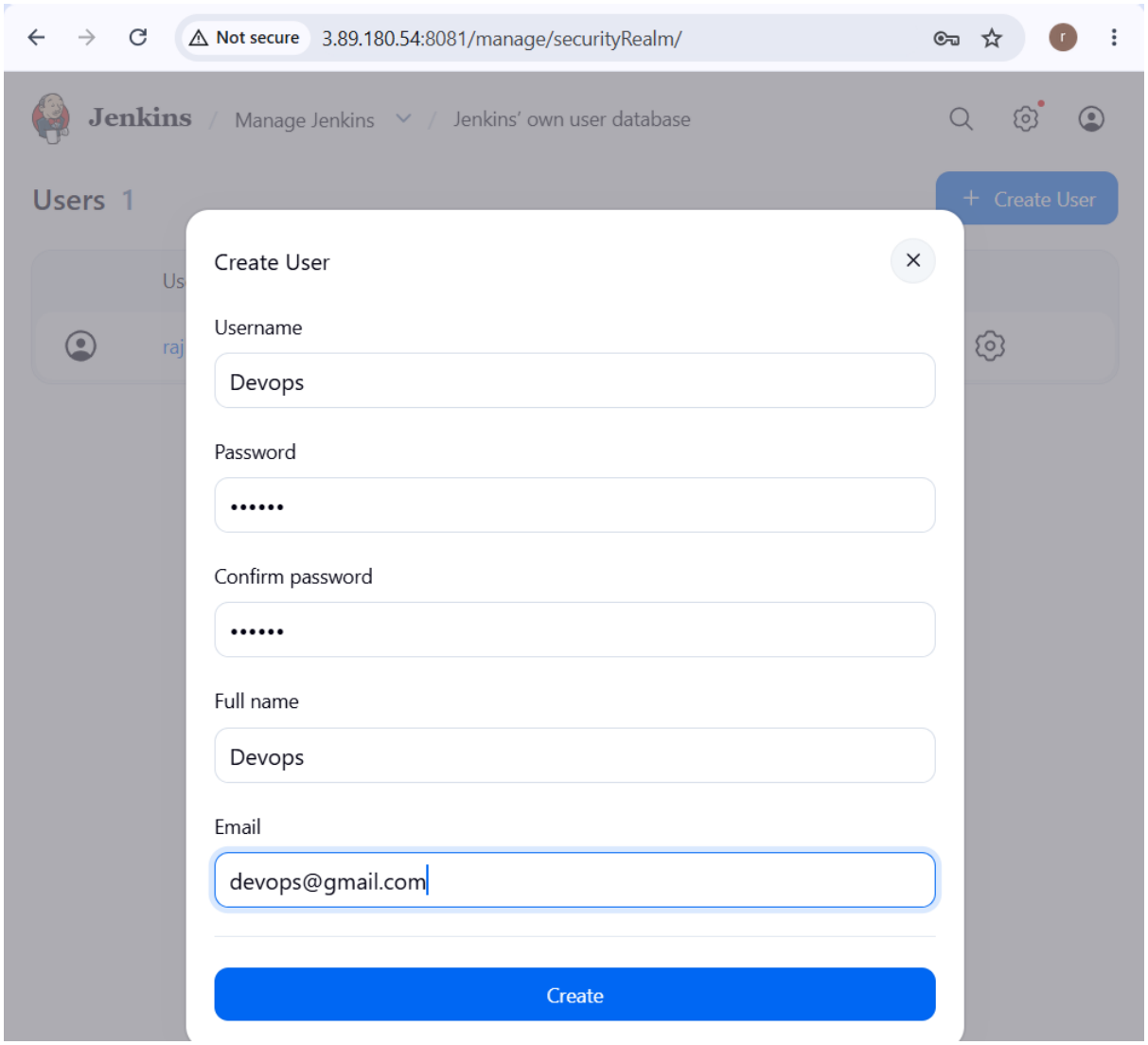
Click:

Create User

Enter the following details:

- **Username:** DevOps
- **Password:** Enter a strong password
- **Confirm password:** Re-enter the password
- **Full name:** DevOps
- **Email address:** Optional

Click **Create User**.



Step 5: Create the Testing User

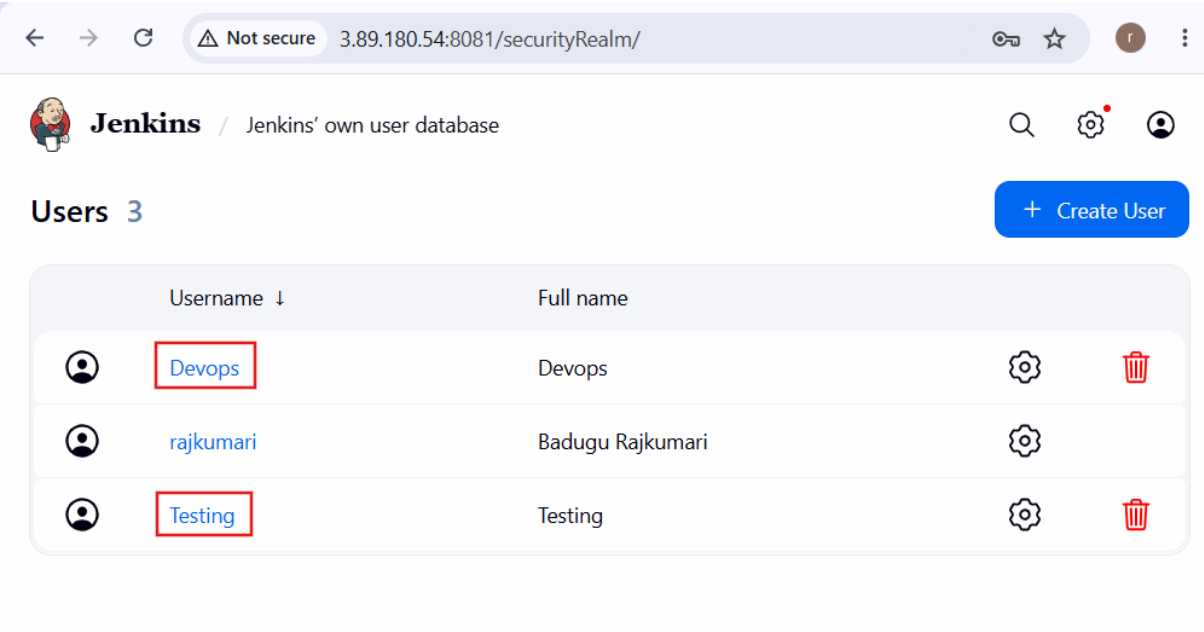
Again, click:

Create User

Enter:

- **Username:** Testing
- **Password:** Enter a strong password
- **Confirm password:** Re-enter the password
- **Full name:** Testing
- **Email address:** Optional

Click **Create User**.



Step 6: Open Security Configuration

Return to:

Manage Jenkins → Security

Locate the **Authorization** section.

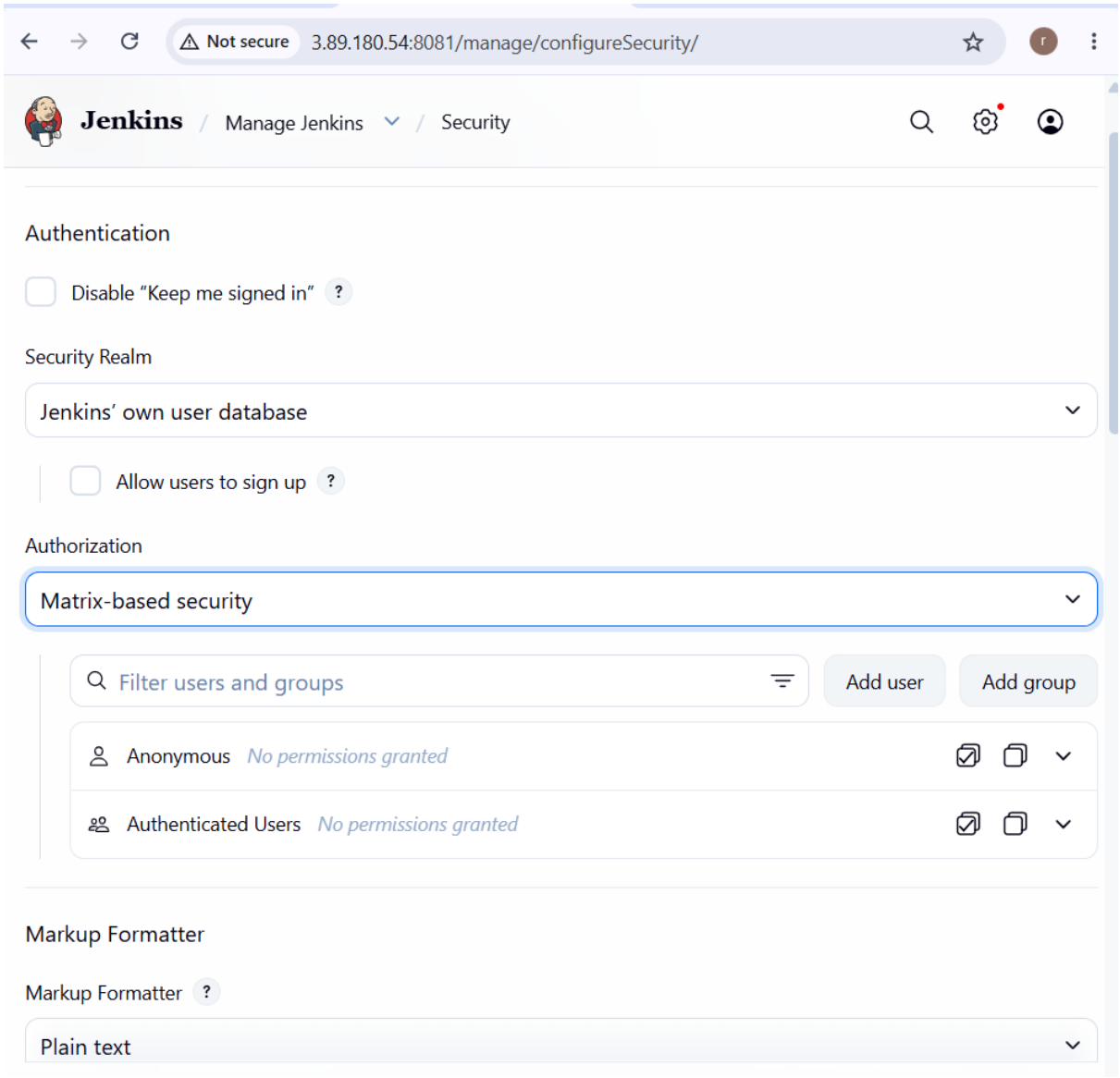
Step 7: Select Matrix-Based Security

Under **Authorization**, select:

Matrix-based security

This allows you to provide different permissions to different Jenkins users.

If **Matrix-based security** is not available, install the **Matrix Authorization Strategy** plugin through **Manage Jenkins → Plugins**.



Step 8: Add the DevOps User

In the authorization matrix, click **Add user or group**.

Enter:

DevOps

Select the DevOps user from the list.

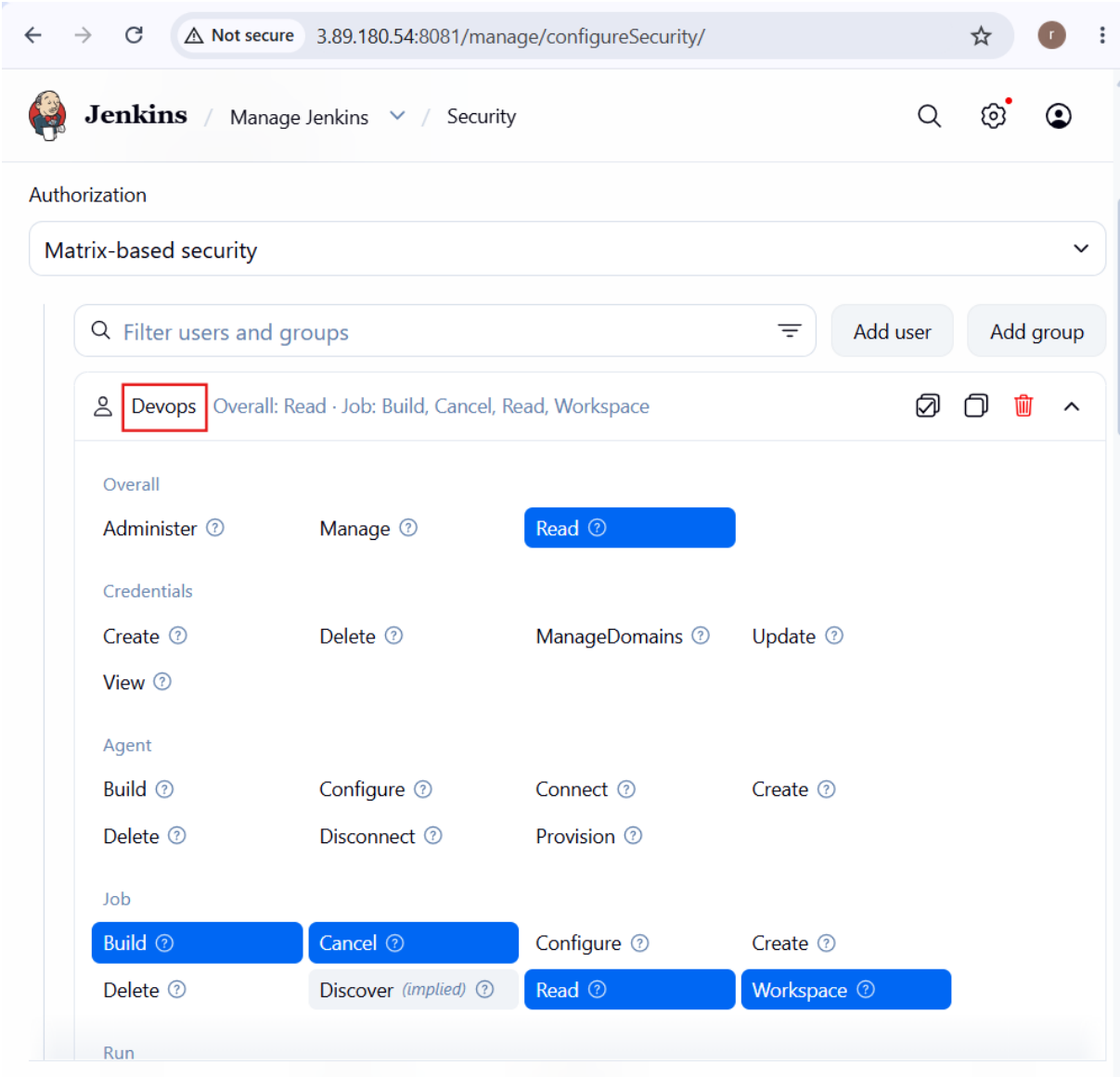
Step 9: Give Limited DevOps Permissions

Grant only the permissions required for DevOps activities.

For example:

- **Overall → Read** ☑

- **Job → Read** ✅
- **Job → Build** ✅
- **Job → Workspace** ✅
- **Job → Cancel** ✅



Step 10: Add the Testing User

Click **Add user or group** again and enter:

Testing

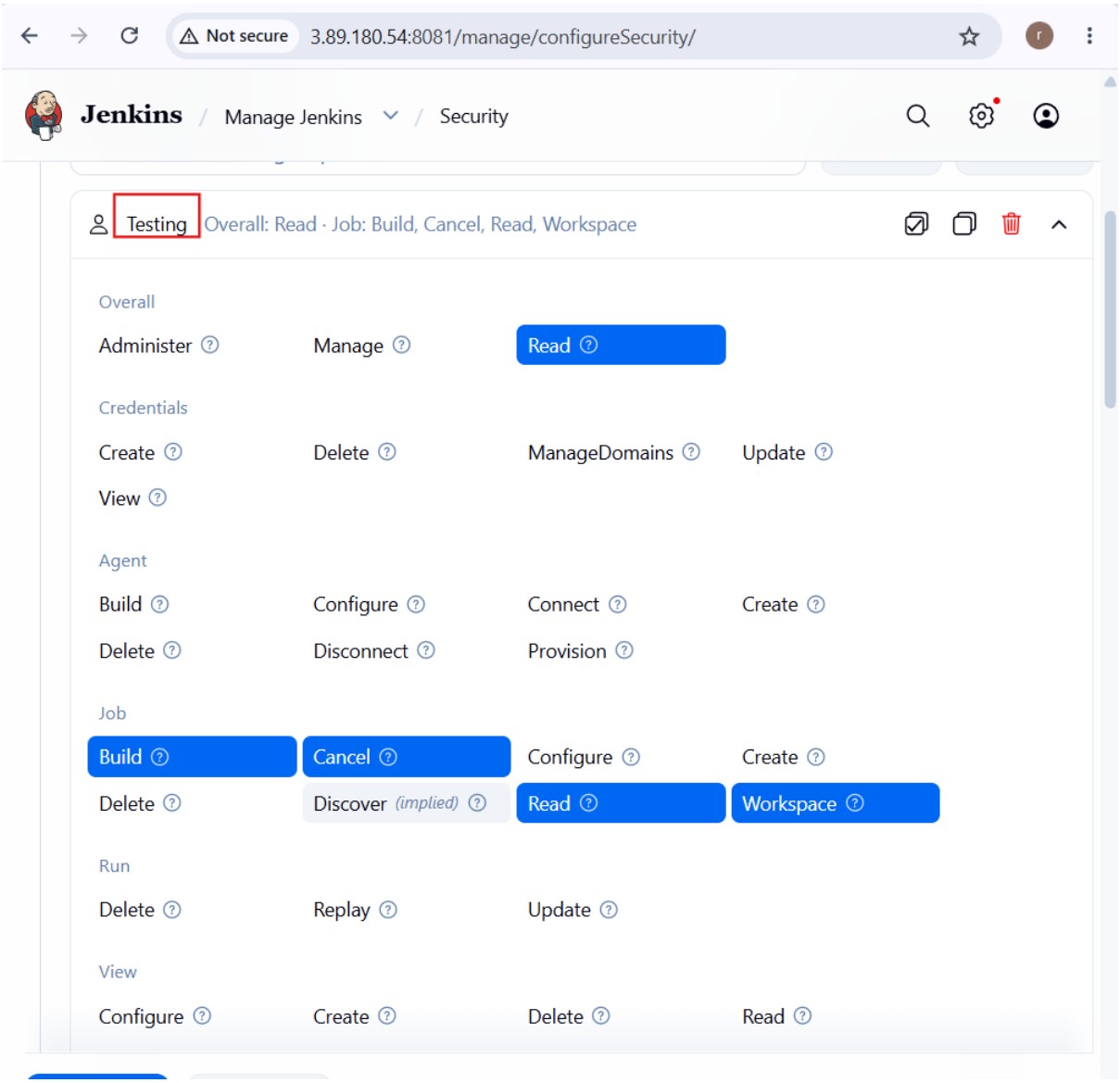
Select the Testing user.

Step 11: Give Limited Testing Permissions

Grant permissions appropriate for testing activities:

- **Overall → Read** ✅
- **Job → Read** ✅
- **Job → Build** ✅
- **Job → Workspace** ✅
- **Job → Cancel** ✅

Do not provide administrative permissions.



Step 12: Verify Administrator Access

Make sure your current administrator account still has:

Overall → Administer ☑

This is important because removing administrator access accidentally can lock you out of Jenkins.

Step 13: Save the Security Configuration

Scroll to the bottom of the page and click:

Save

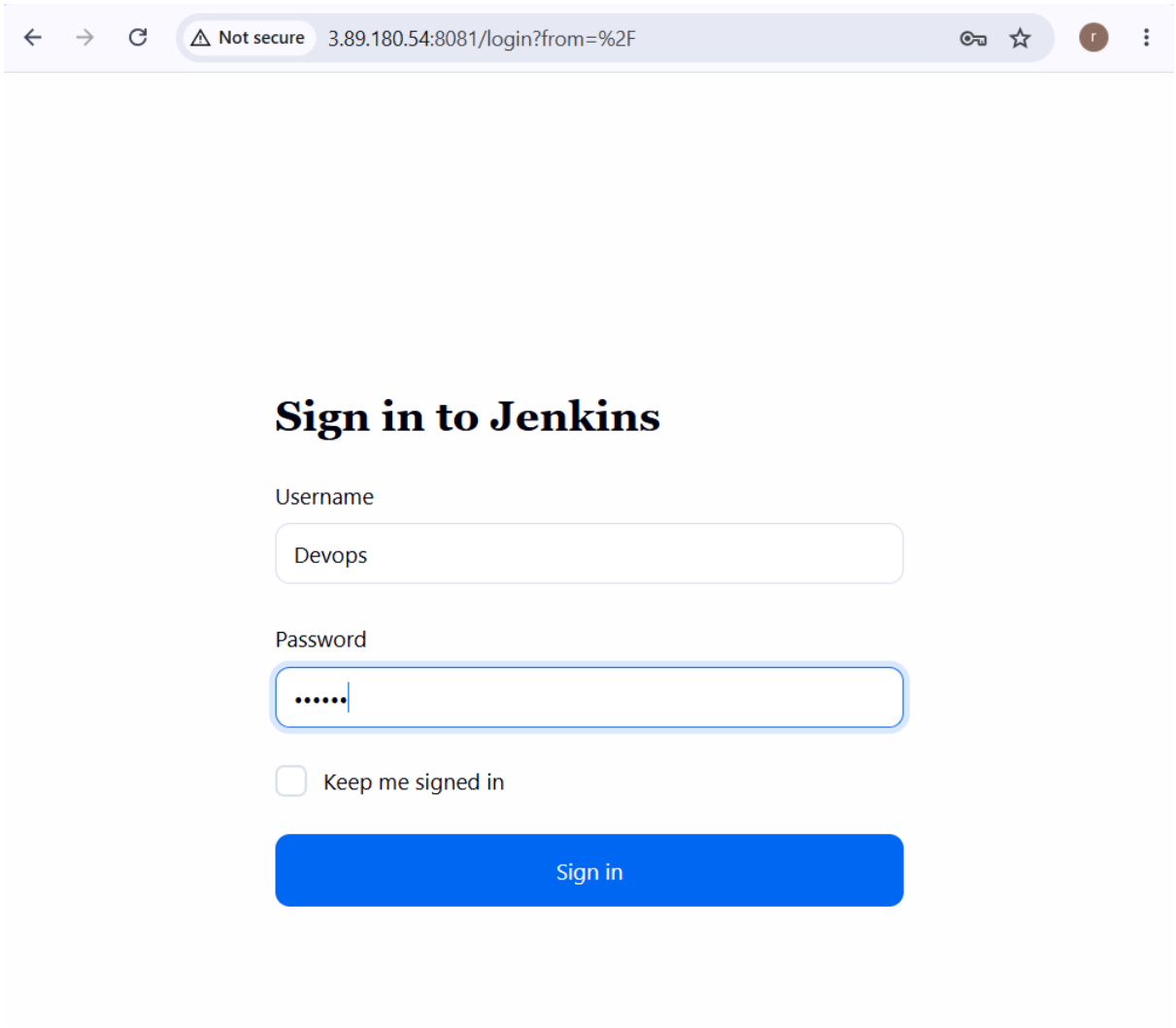
Jenkins will now apply the new authorization settings.

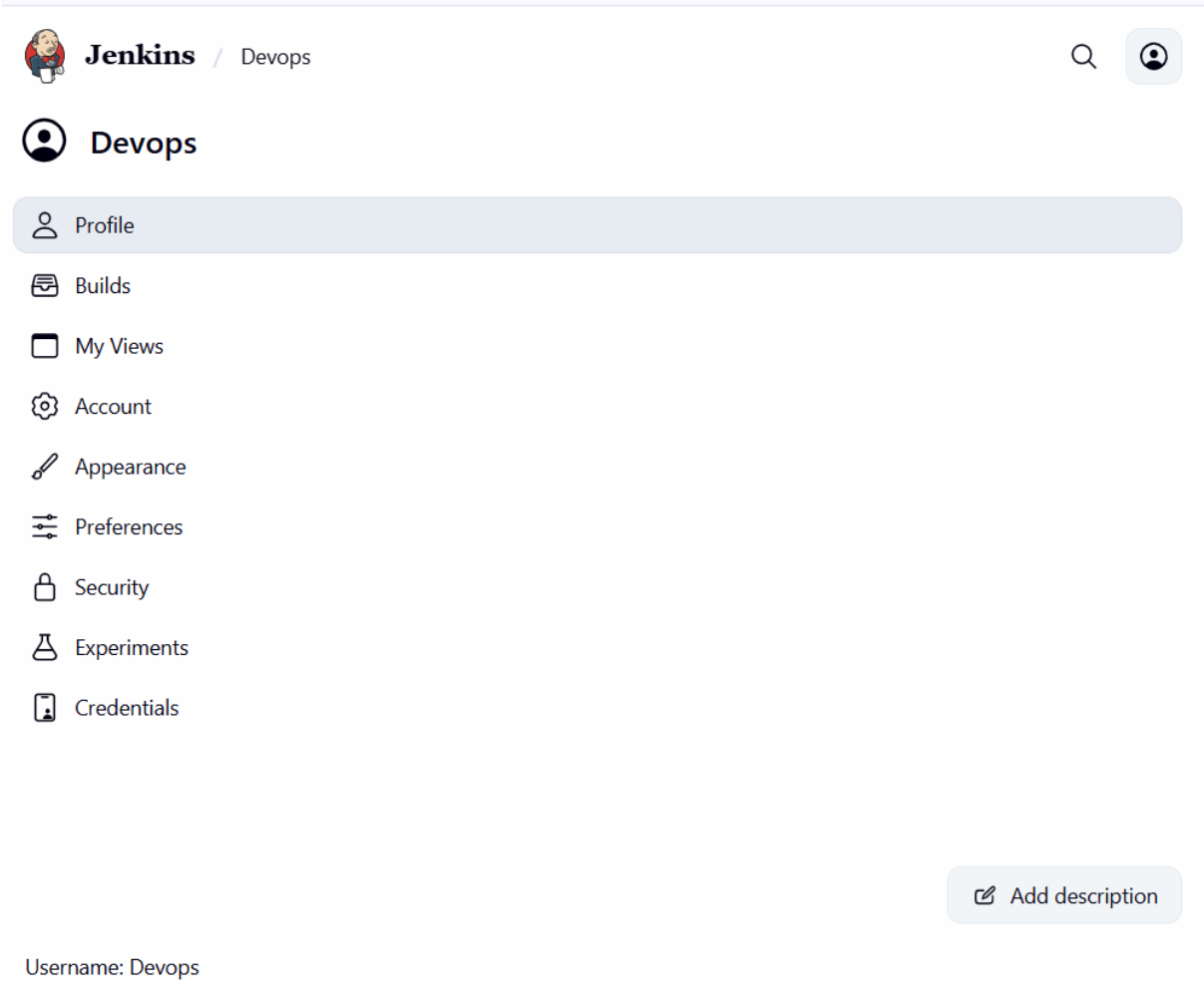
Step 14: Test the DevOps User

Log out from the administrator account and log in as:

DevOps

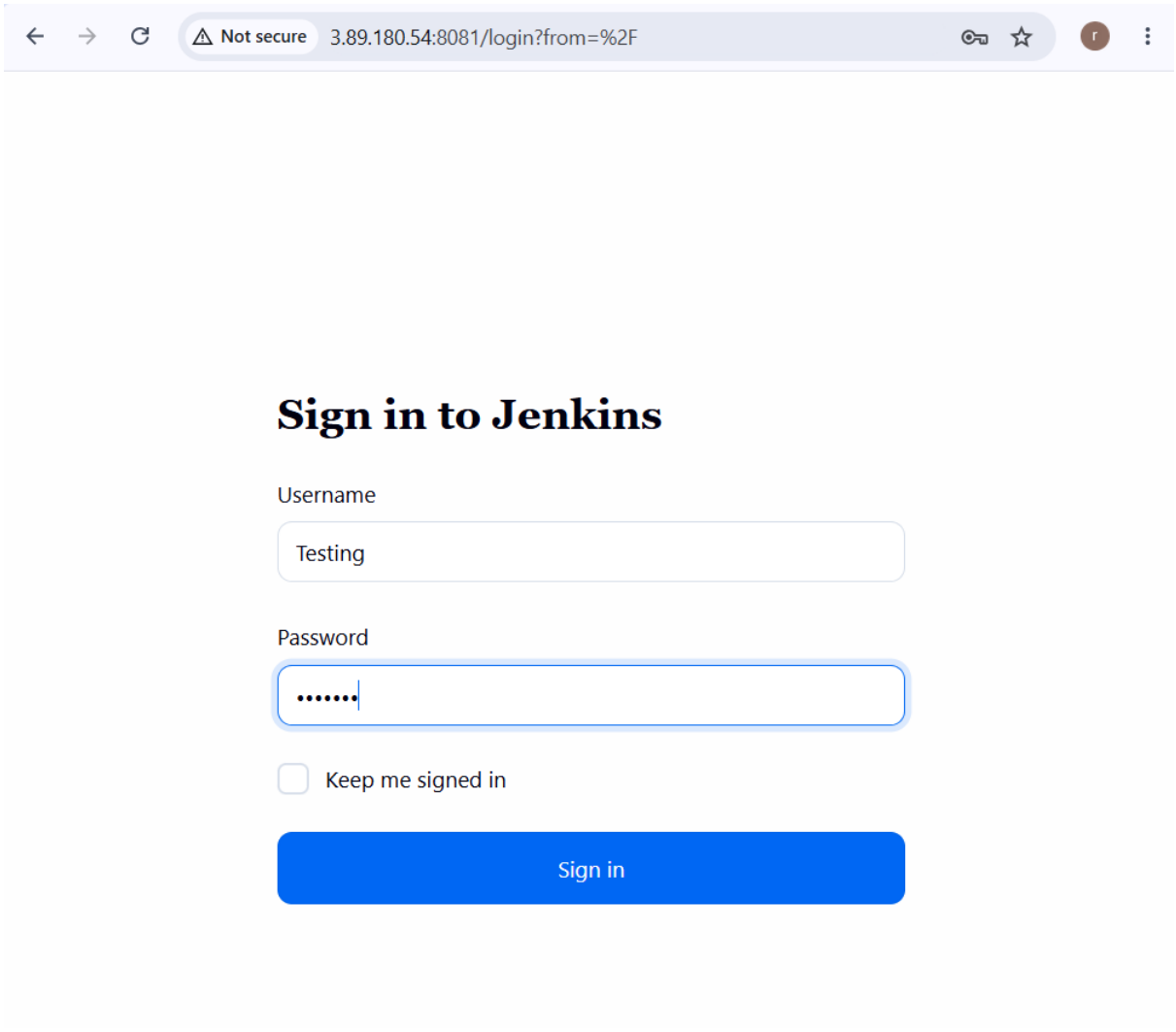
Verify that the user can access and build the required Jenkins jobs but **cannot access administrative configuration**.

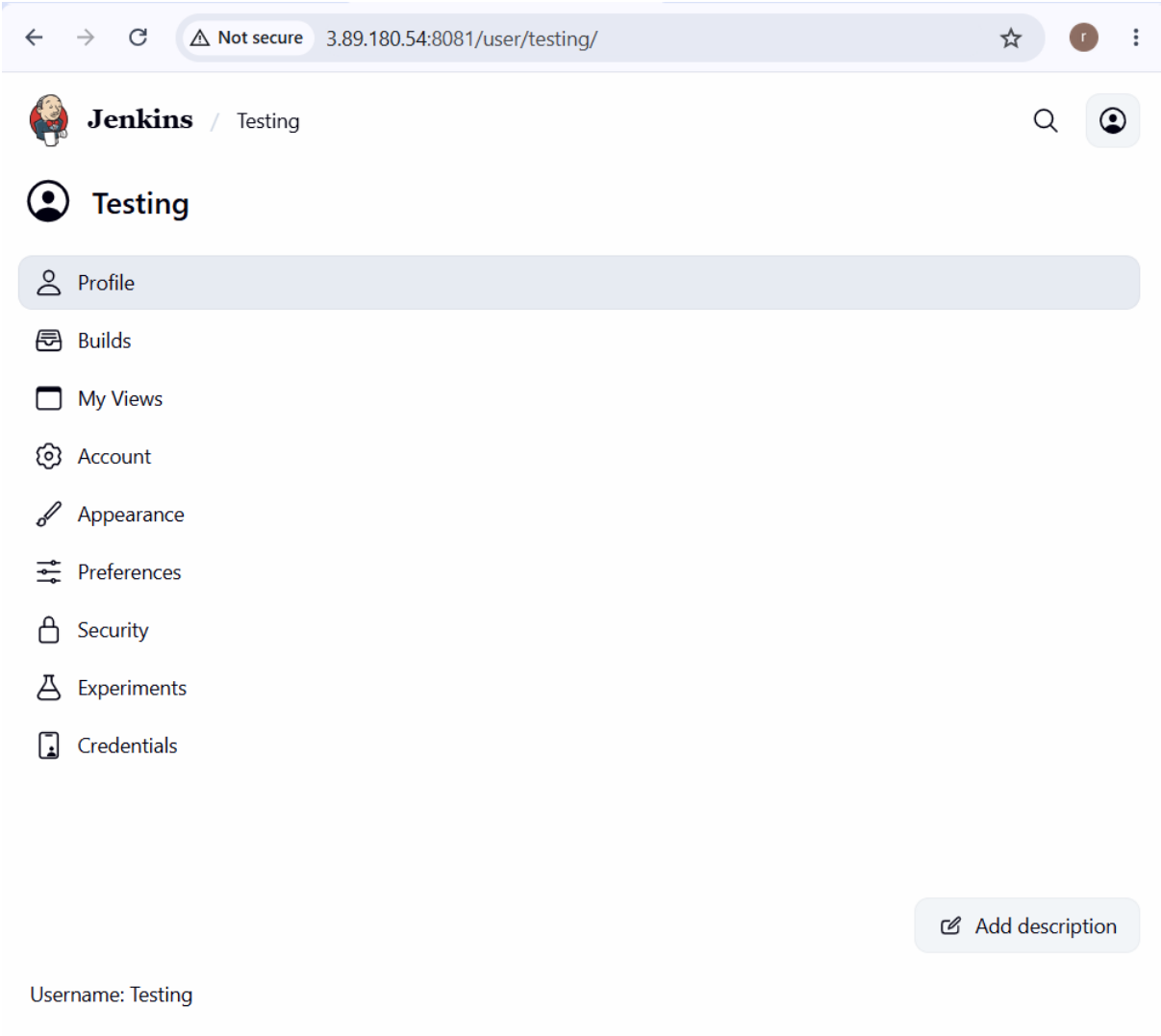

Not secure
3.89.180.54:8081/login?from=%2F
Sign in to Jenkins
Username
Devops
Password
Keep me signed in
Sign in



Step 15: Test the Testing User

Log out and log in as: Testing


Not secure
3.89.180.54:8081/login?from=%2F
Sign in to Jenkins
Username
Testing
Password
Keep me signed in
Sign in



Verify that the user can view and execute the required jobs but cannot perform administrative operations such as changing Jenkins security settings, managing users, or configuring credentials.

Expected Result

User	Access Level	Main Permissions
Administrator	Full Access	Manage Jenkins, users, security, credentials, jobs
DevOps	Limited Access	Read, Build, Workspace, Cancel
Testing	Limited Access	Read, Build, Workspace, Cancel

4. Configure labels and restrict the jobs to execute based on label only.

The goal is to create labels such as devops-node and testing-node and ensure that a job runs **only on the node matching its configured label**.

Step 1: Open Jenkins

Log in to Jenkins using an account with **Administrator** privileges.

Step 2: Open Manage Jenkins

From the Jenkins Dashboard, click:

Manage Jenkins

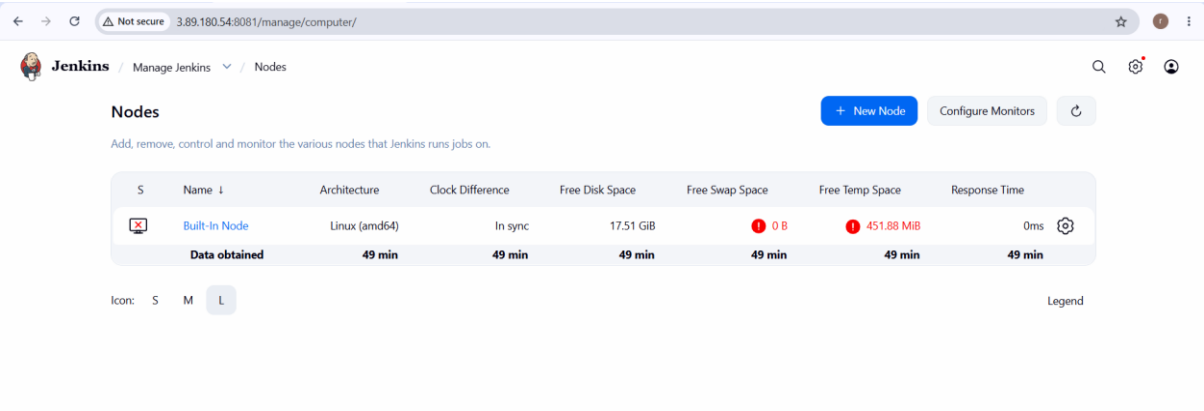
Step 3: Open Nodes

Click:

Nodes

Depending on your Jenkins version, this may appear as:

Manage Jenkins → Nodes



Step 4: Select the Jenkins Node

Select the node on which you want to run the job.

For example:

Built-In Node

or an existing agent such as:

DevOps-Agent

Step 5: Open Node Configuration

Click:

Configure

This opens the configuration page for the selected node.

Step 6: Configure the Node Label

Locate the:

Labels

field.

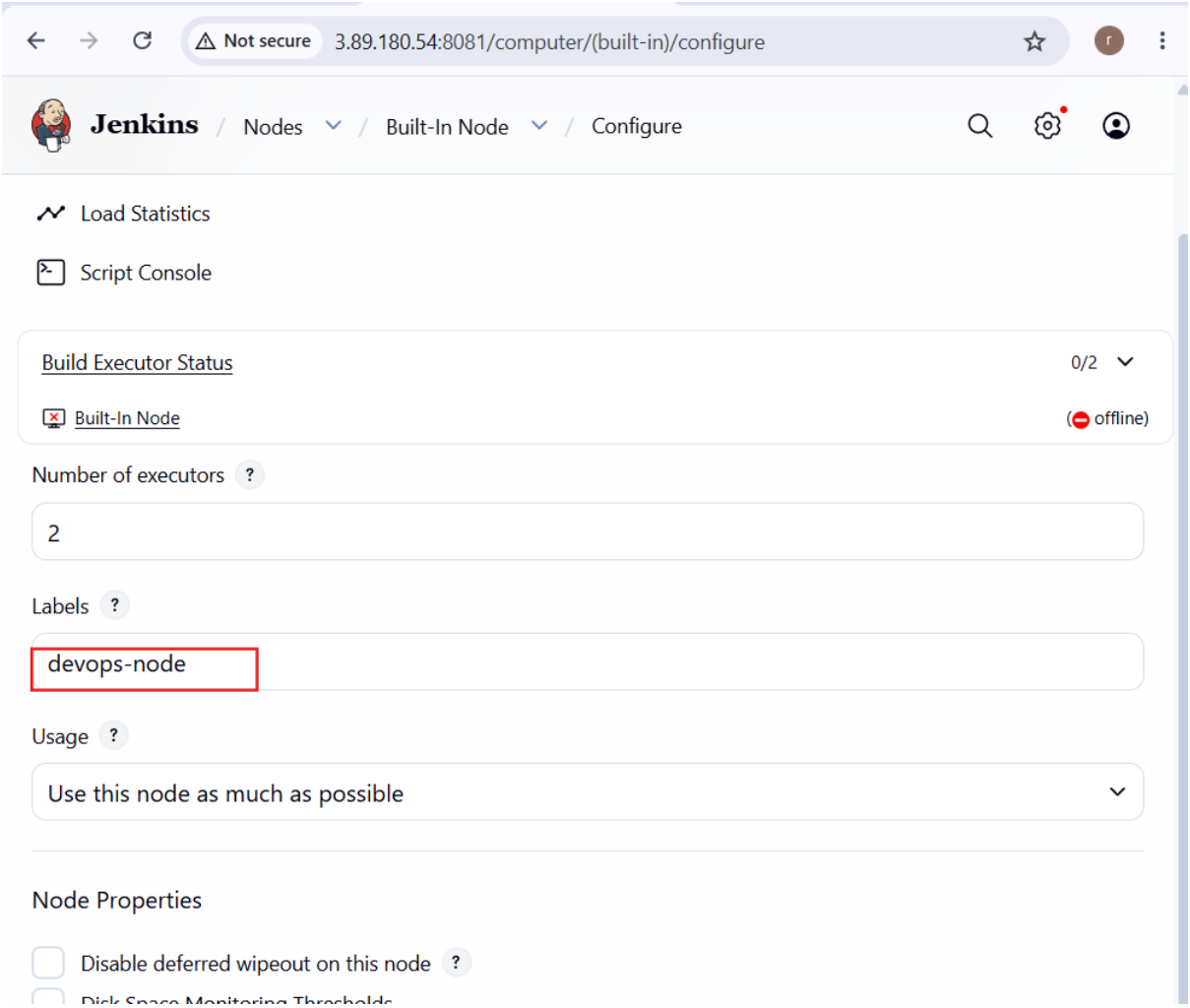
Enter a label such as:

devops-node

If you have a separate testing agent, configure its label as:

testing-node

Labels are used by Jenkins to identify which node should execute a particular job.



Step 8: Create or Open a Jenkins Job

Go back to the Jenkins Dashboard.

Create a new **Freestyle project** or open an existing job.

For example:

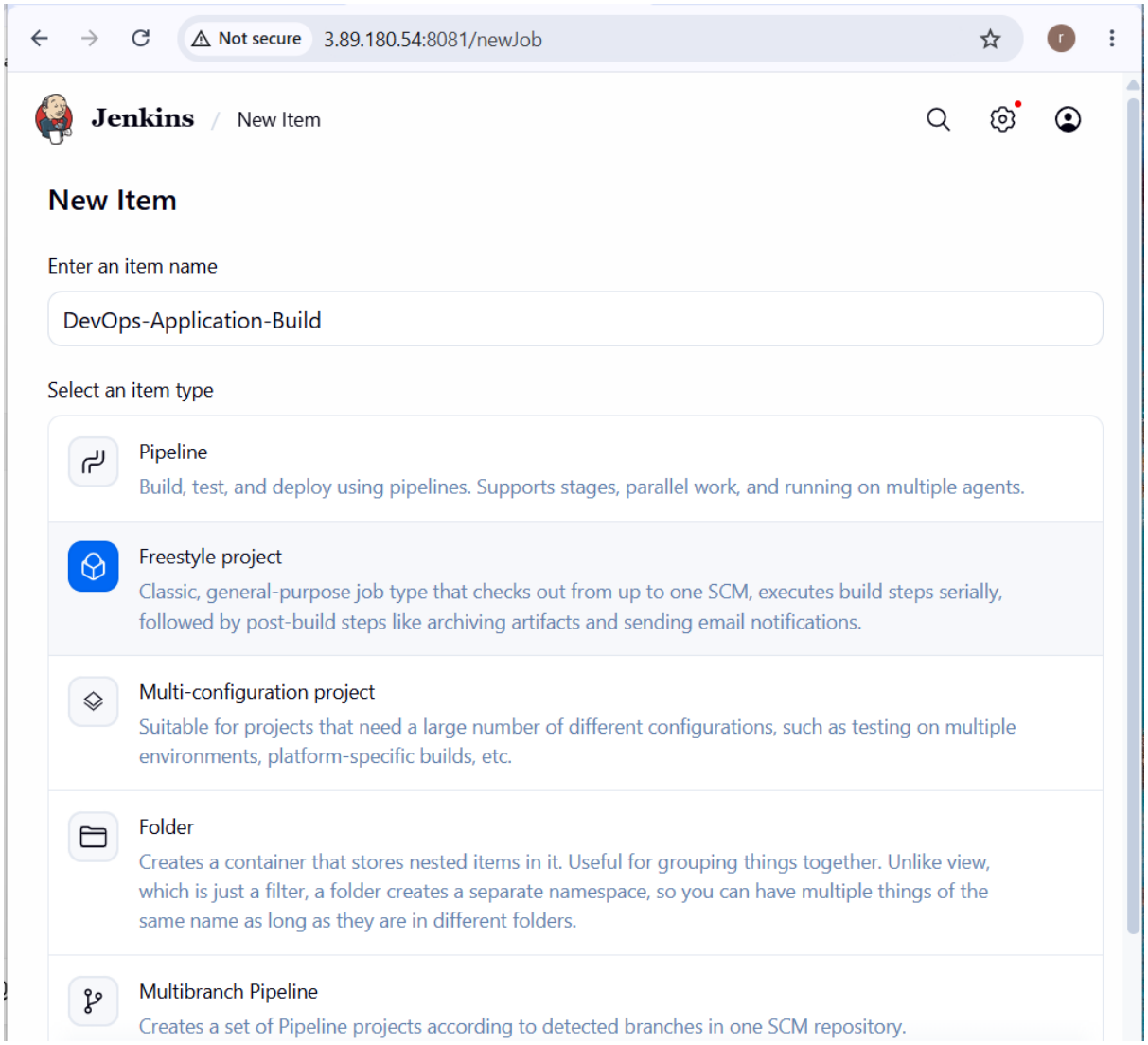
DevOps-Application-Build

Step 9: Open Job Configuration

Inside the job, click:

Configure

Scroll to the **General** section.



Step 10: Enable Label Restriction

Find and select:

Restrict where this project can be run

A text box called **Label Expression** will appear.

Step 11: Enter the Required Label

Enter:

devops-node

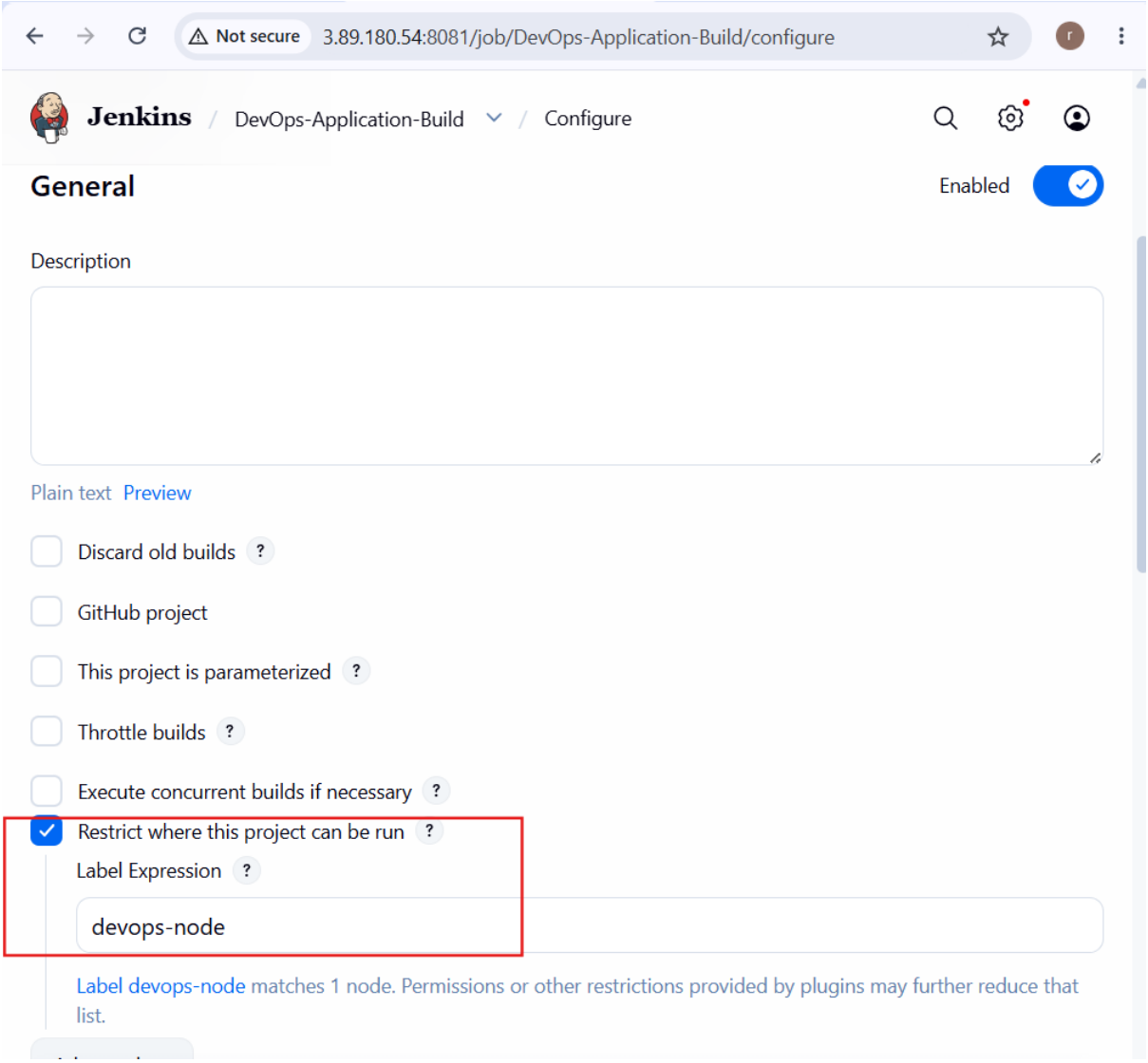
This tells Jenkins that the job must execute on a node having the devops-node label.

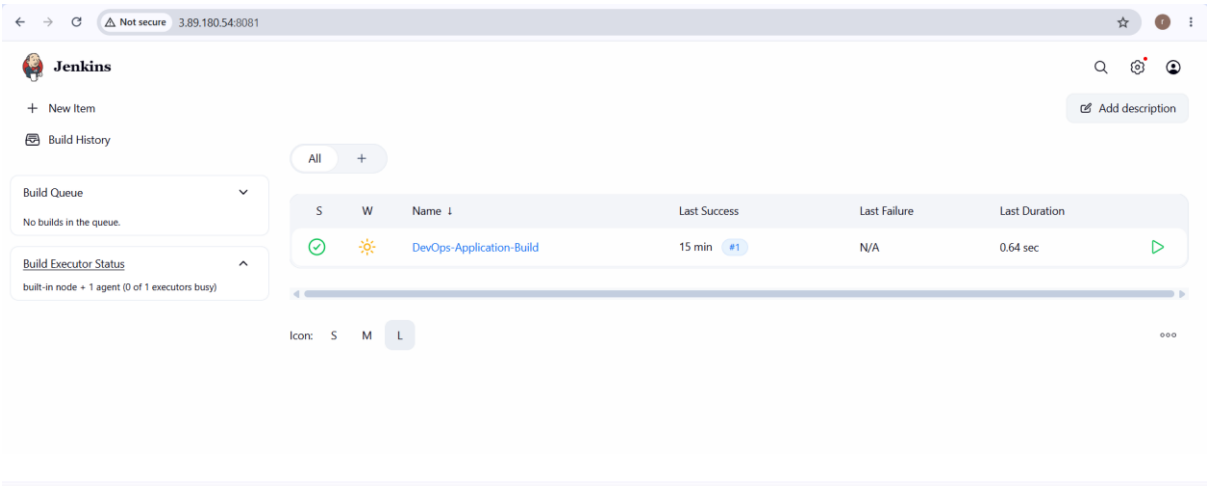
Step 12: Save the Job

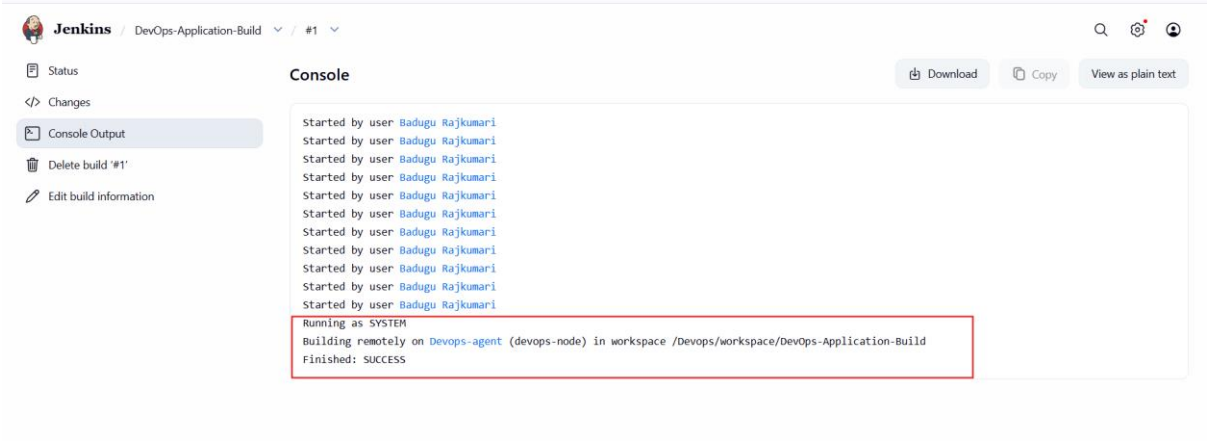
Click:

Save

The job is now restricted to the specified label.







5. Create Three sample jobs using the below URL.

- ***Source Code:*** https://github.com/betawins/Techie_horizon_Login_app.git

NOTE: First, Need to Install git to our EC2

Job	Purpose
DevOps-App-Build	Clone and build/check the application
DevOps-App-Test	Clone and test the application
DevOps-App-Deploy	Clone and prepare/deploy the application

Job 1 — DevOps-App-Build

Step 1

Go to: **Jenkins Dashboard → New Item**

Step 2

Enter: DevOps-Application-Build

Select: **Freestyle project**

Click **OK**.

Step 3 — Restrict the job to your agent

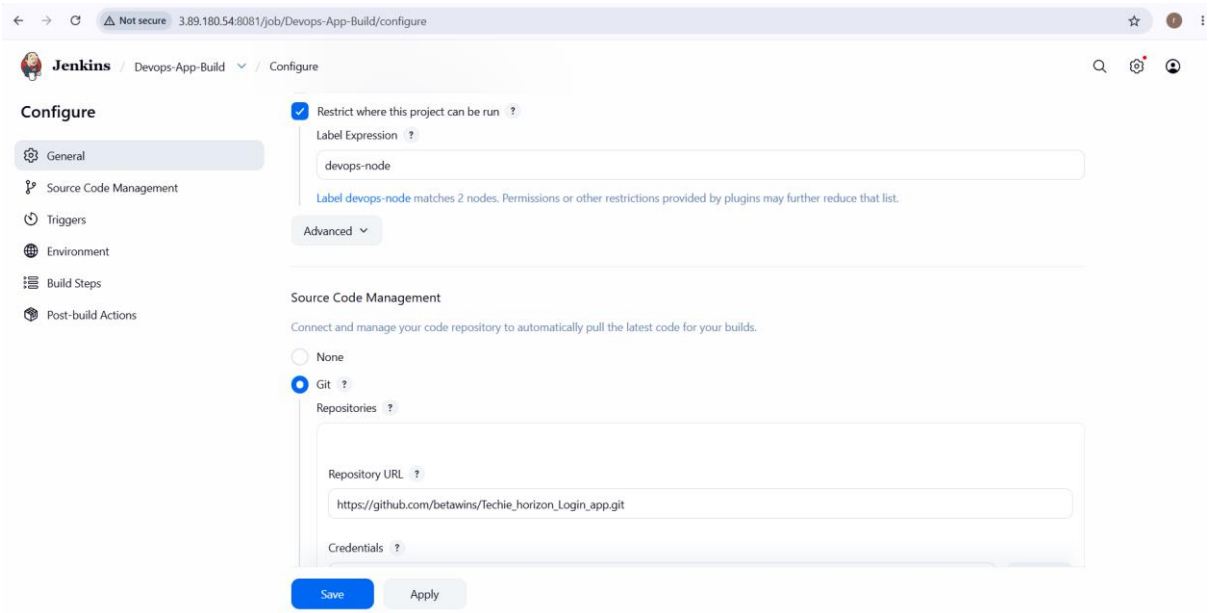
Under **General**, select:

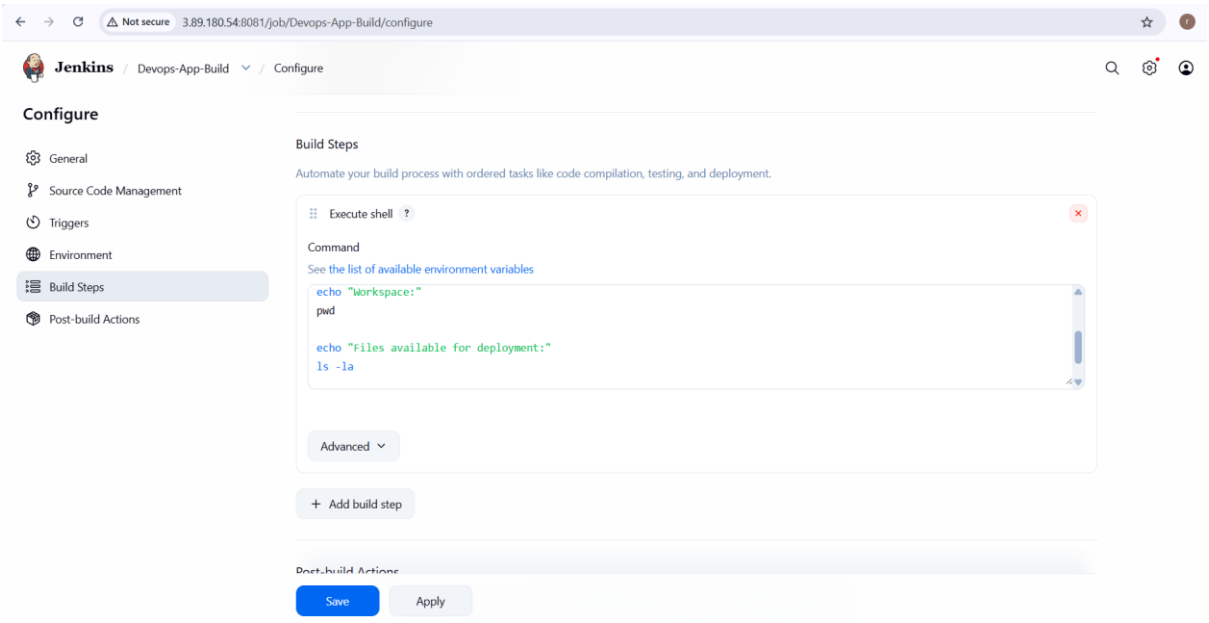
Restrict where this project can be run

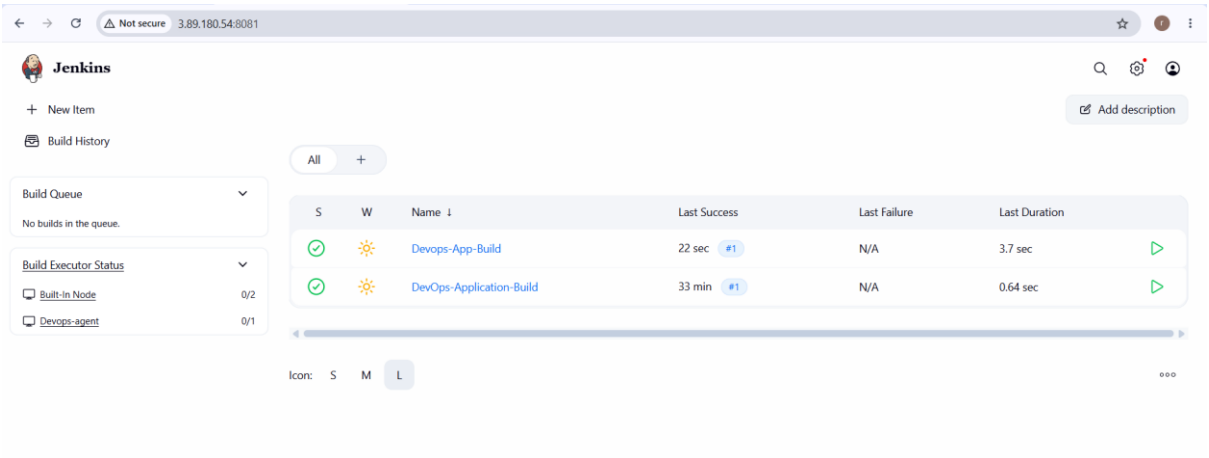
Enter:

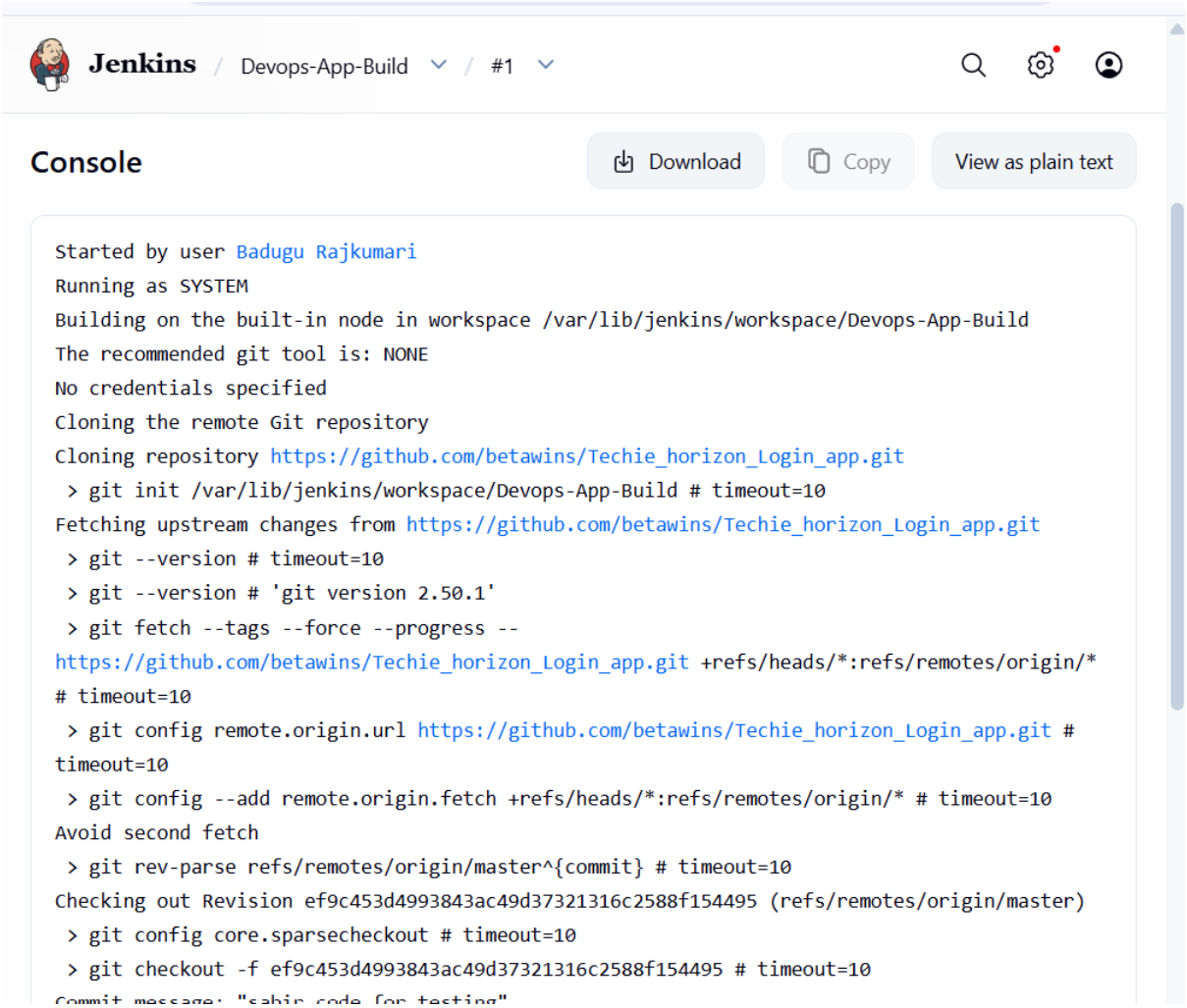
devops-node

This ensures the job runs on your Devops-agent.




Not secure 3.89.180.54:8081/job/Devops-App-Build/configure
Jenkins / Devops-App-Build / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Build Steps
Automate your build process with ordered tasks like code compilation, testing, and deployment.
Execute shell
Command
See the list of available environment variables
echo "Workspace:"
pwd
echo "Files available for deployment:"
ls -la
Advanced
Add build step
Post-build Actions
Save
Apply


Not secure 3.89.180.54:8081
Jenkins
New Item
Build History
Add description
All
Build Queue
No builds in the queue.
Build Executor Status
Built-In Node 0/2
Devops-agent 0/1
S W Name ↓ Last Success Last Failure Last Duration
Devops-App-Build 22 sec #1 N/A 3.7 sec
DevOps-Application-Build 33 min #1 N/A 0.64 sec
Icon: S M L



Job 2 — DevOps-Application-Test

Go to:

Jenkins Dashboard → New Item

Create:

DevOps-Application-Test

Select:

Freestyle project → OK

Step 1 — Agent

Enable:

Restrict where this project can be run

Enter: devops-node

Step 2 — Git

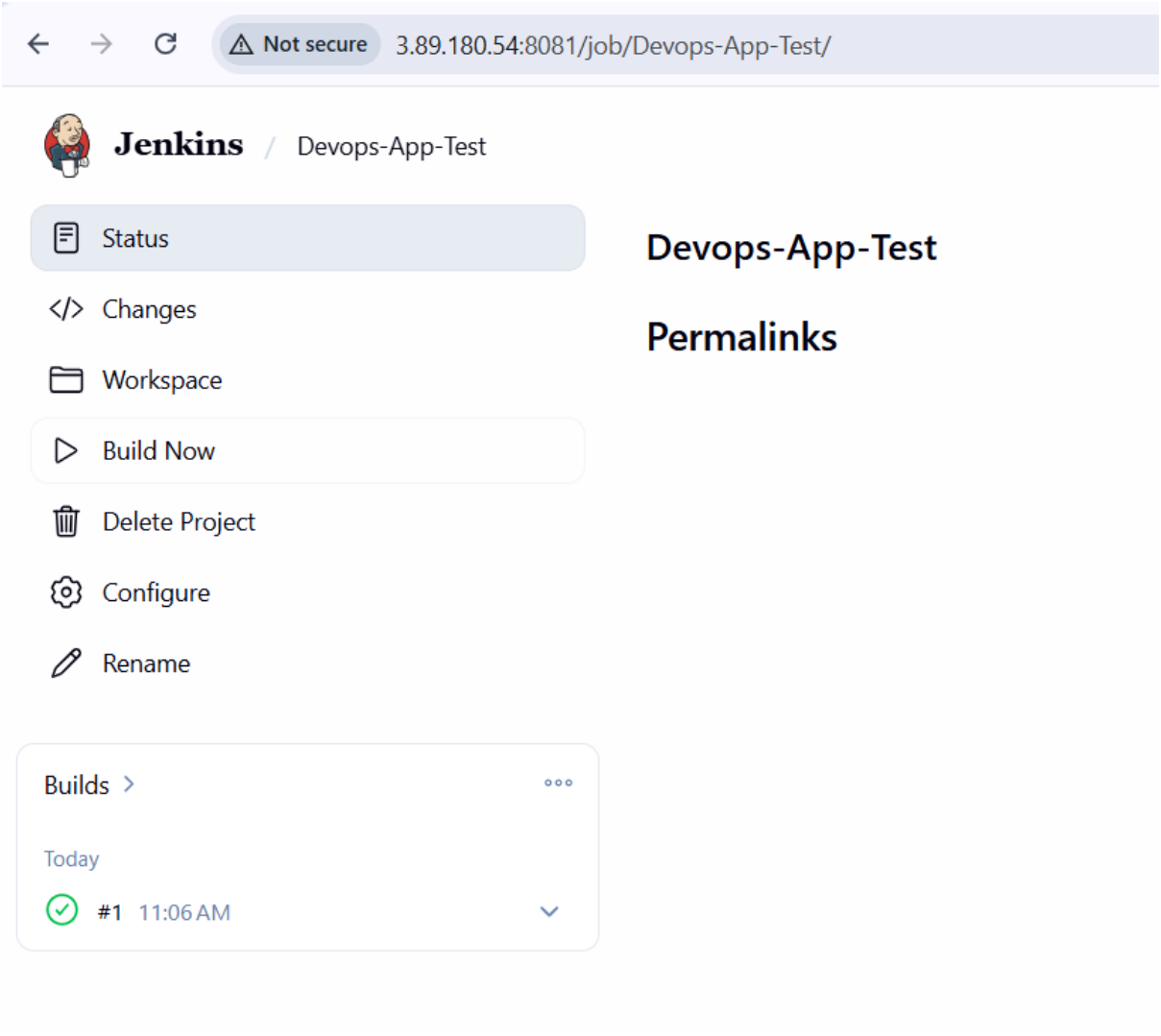
Select:

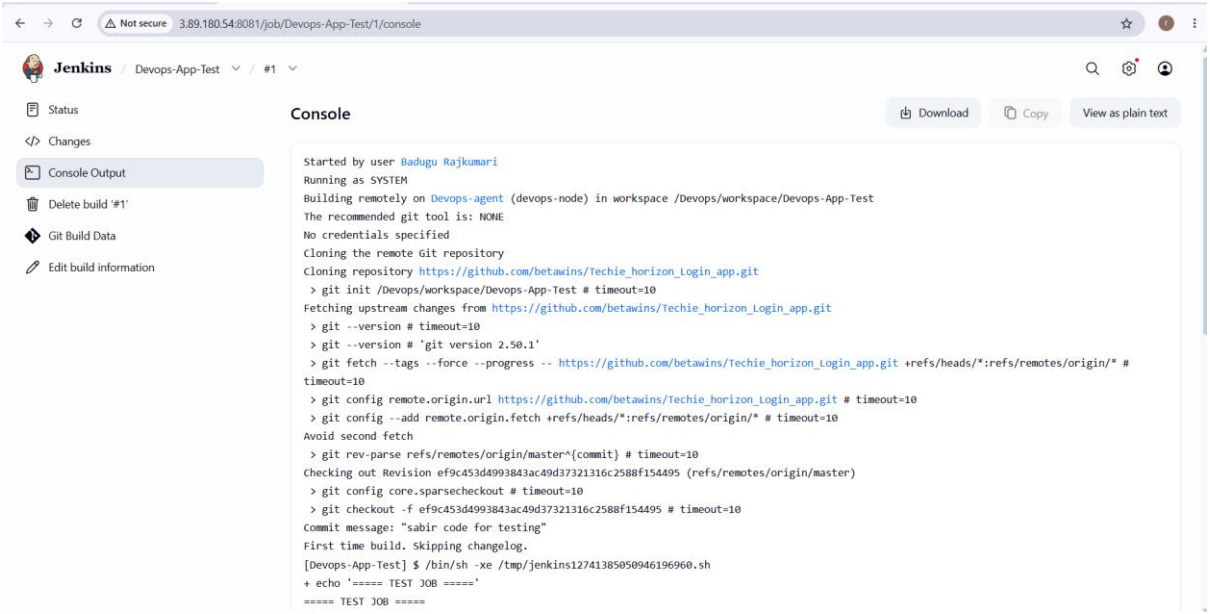
Source Code Management → Git

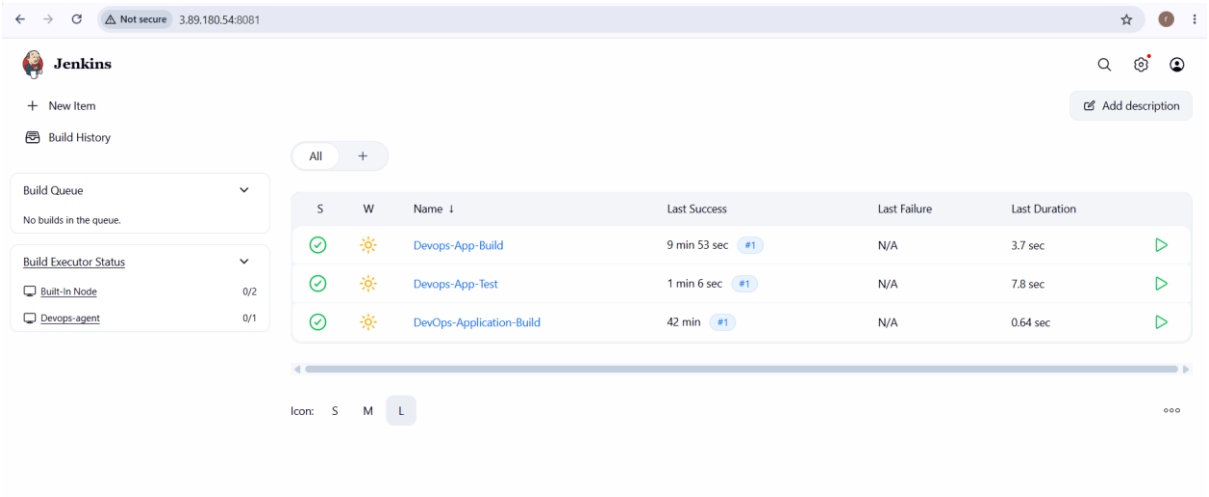
Repository:

https://github.com/betawins/Techie_horizon_Login_app.git

Branch: */master







Job 3 — DevOps-Application-Deploy

Create another:

Jenkins Dashboard → New Item

Name:

DevOps-Application-Deploy

Select:

Freestyle project

Click **OK**.

Step 1 — Agent

Enable:

Restrict where this project can be run

Enter:

devops-node

Step 2 — Git

Repository:

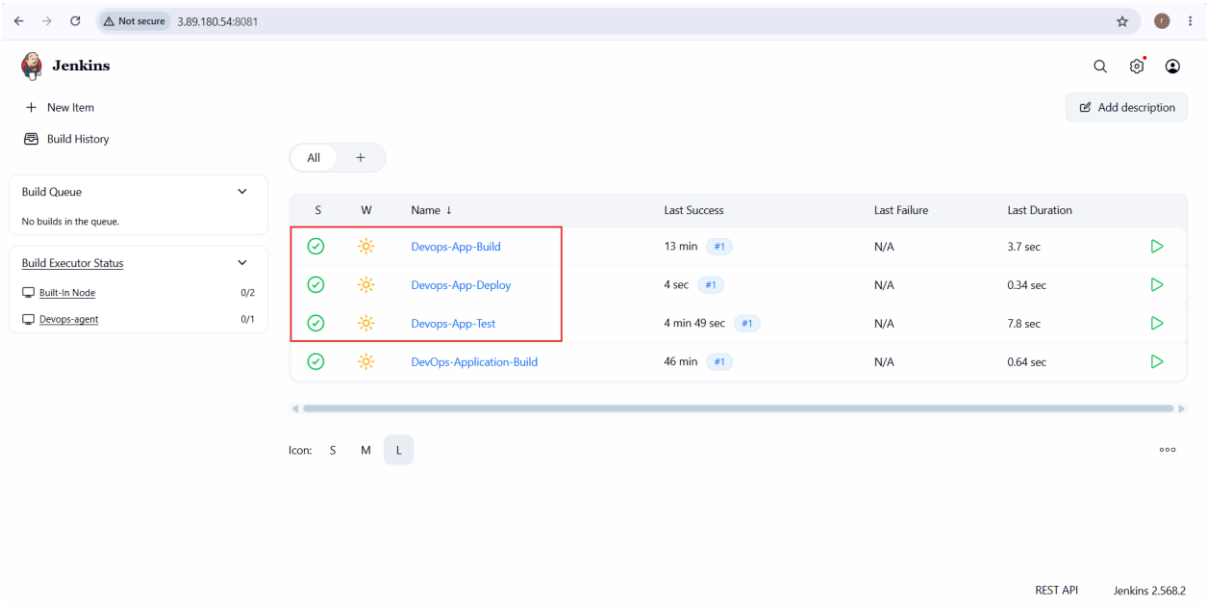
https://github.com/betawins/Techie_horizon_Login_app.git

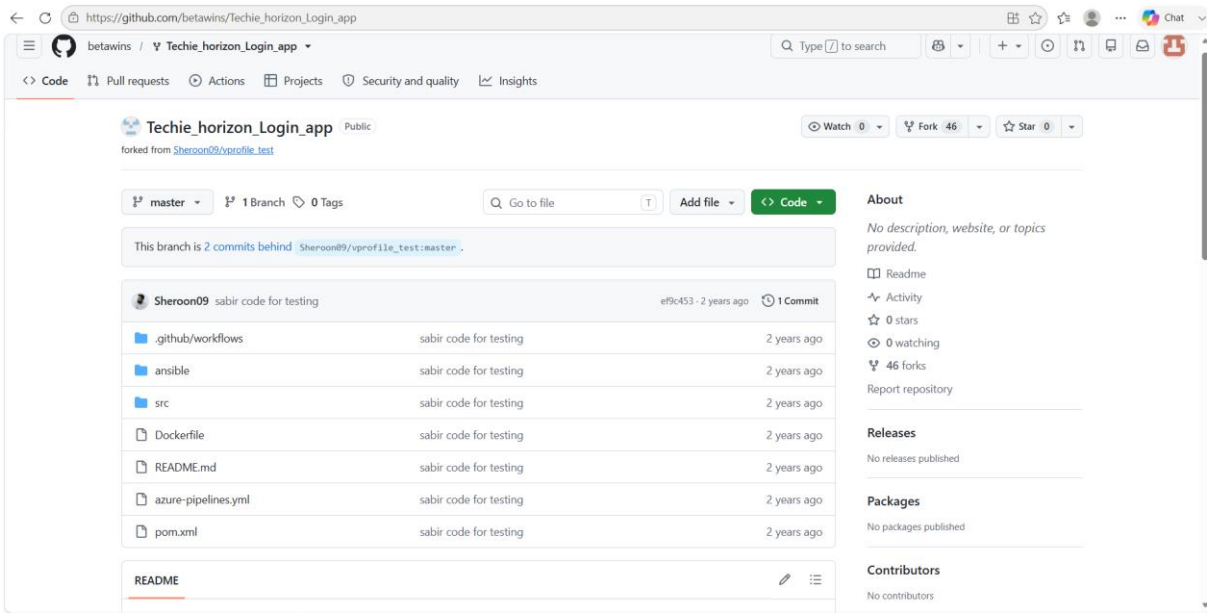
Branch:

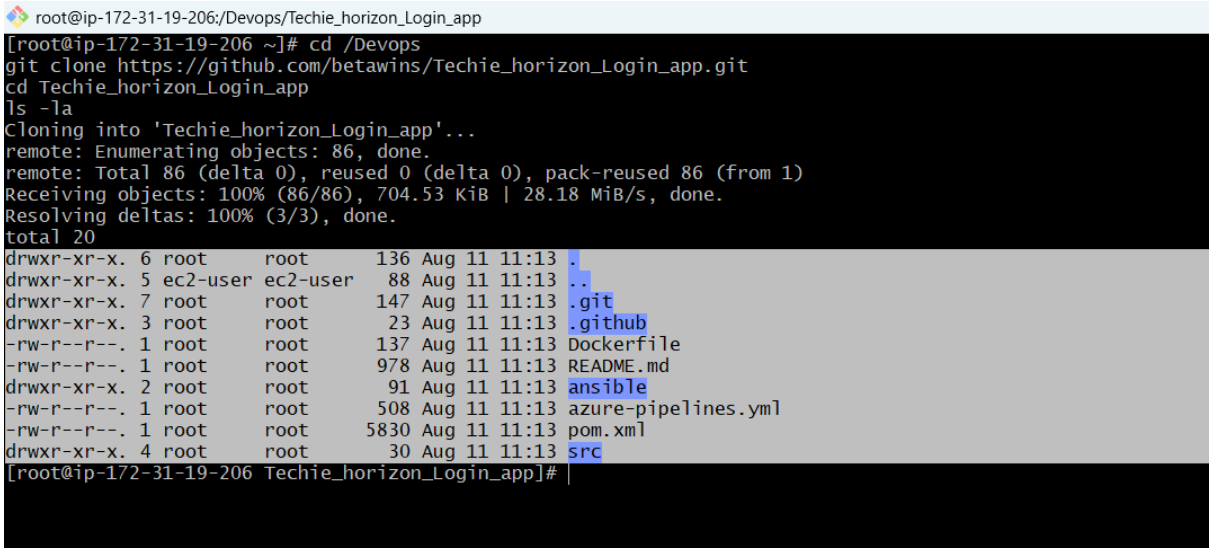
*/main

Step 3 — Execute shell

For your practice documentation, initially use:







6.Create one Jenkins job using GitHub Private repository.

1. Create a Private GitHub Repository

In GitHub:

1. Sign in to GitHub.
2. Click **New repository**.
3. Enter a repository name, for example:

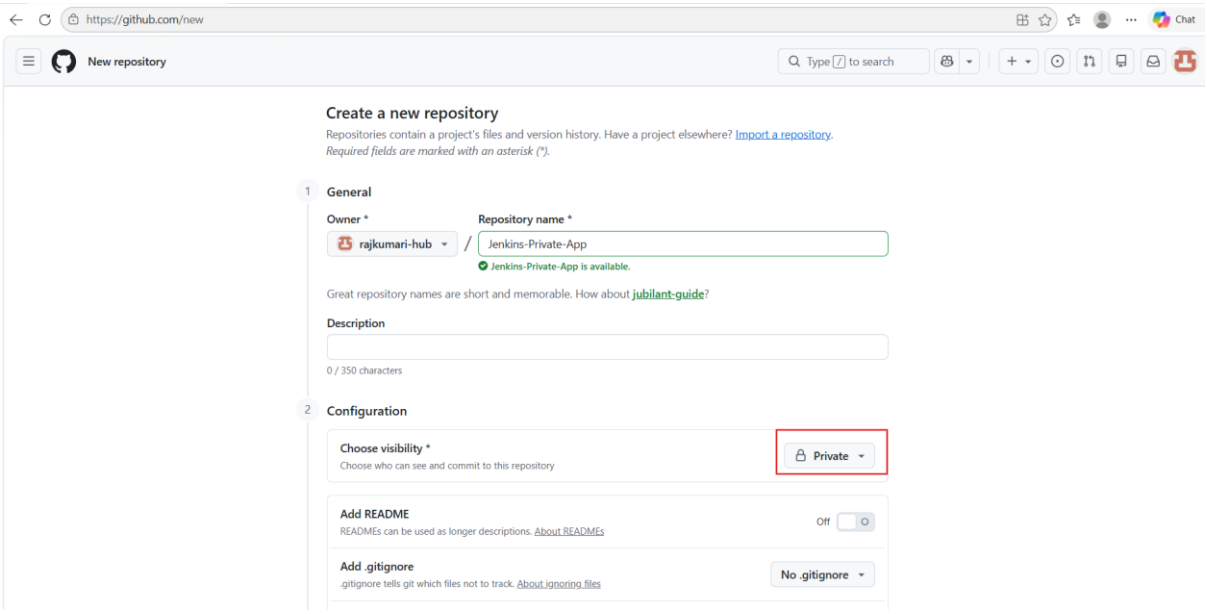
Jenkins-Private-App

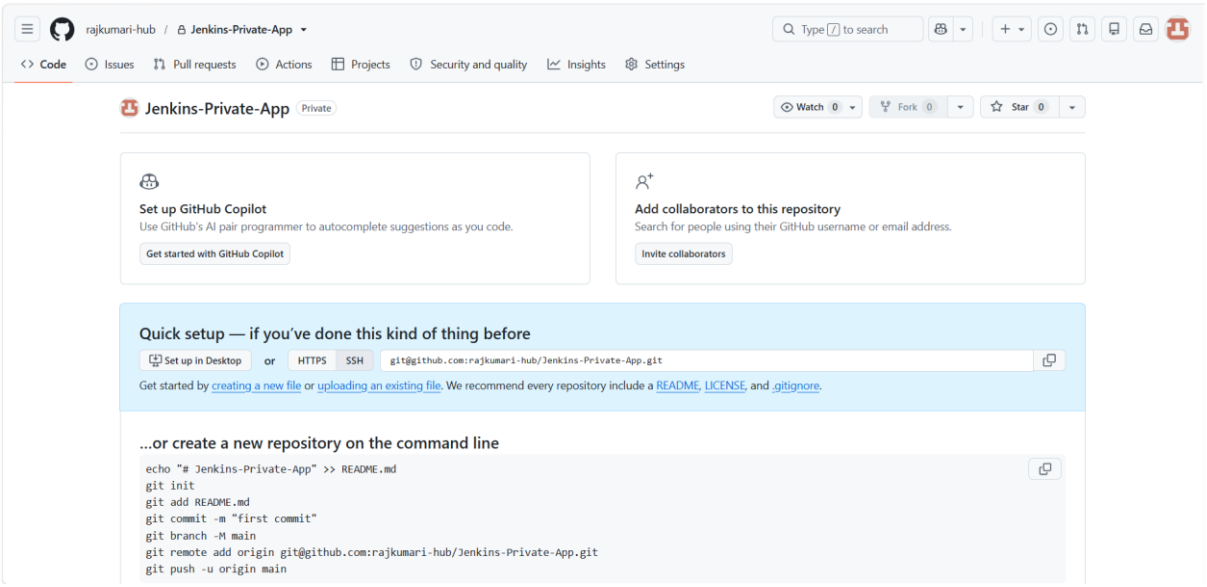
4. Select **Private**.
5. Click **Create repository**.

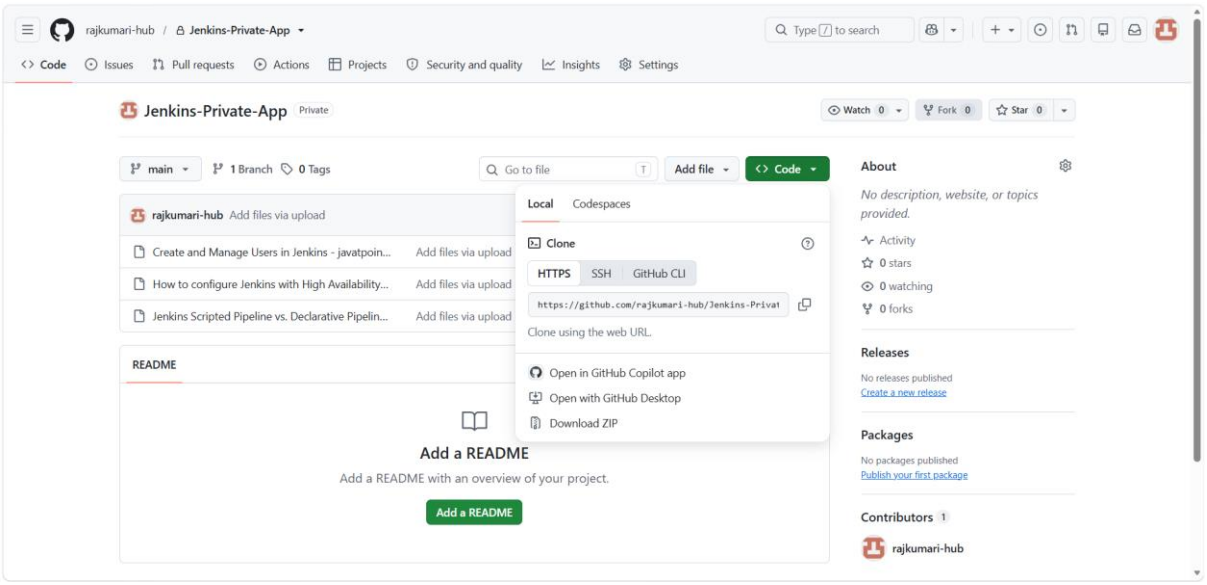
6. Upload your application files to the private repository, or push your existing application.

Your repository URL will look like:

https://github.com/<your-username>/Jenkins-Private-App.git







2. Create GitHub Personal Access Token

For a private repository, Jenkins needs authentication.

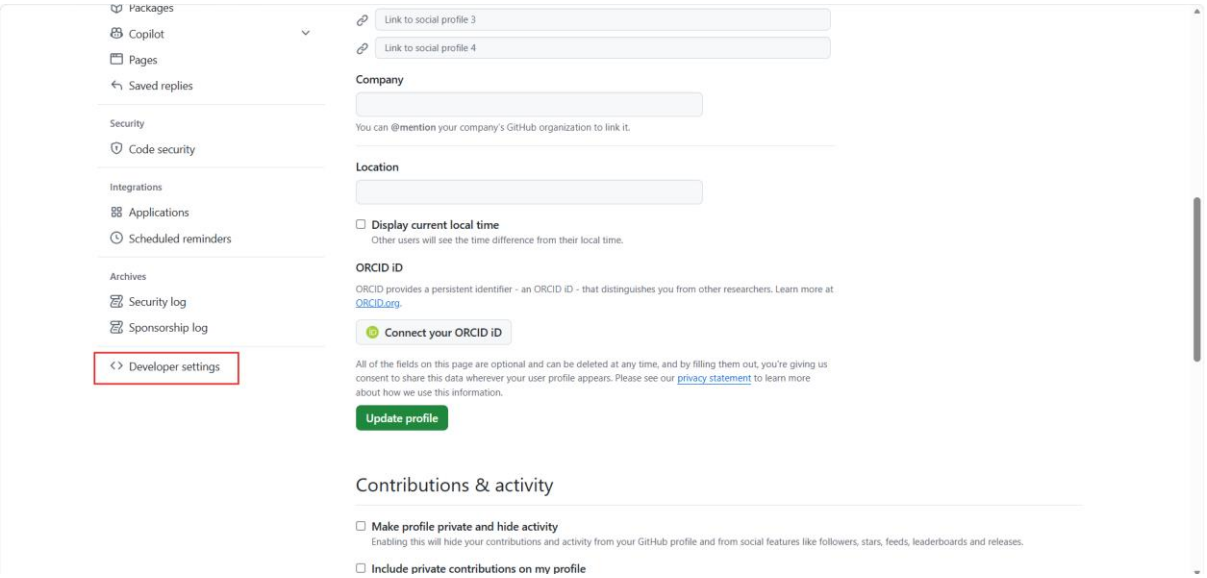
In GitHub:

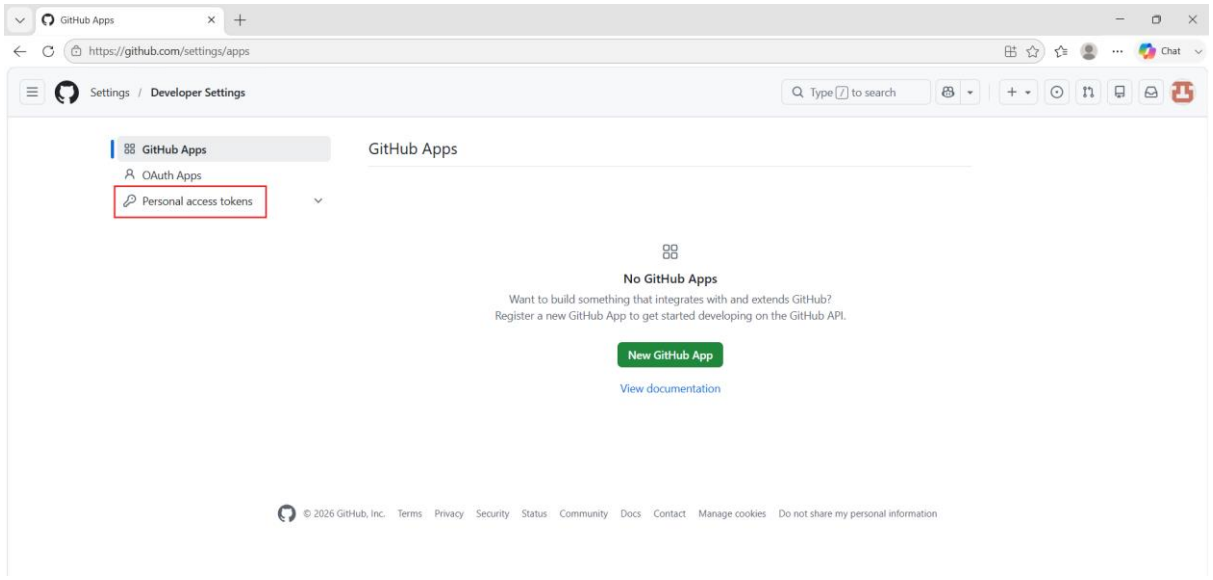
Profile → Settings → Developer settings → Personal access tokens

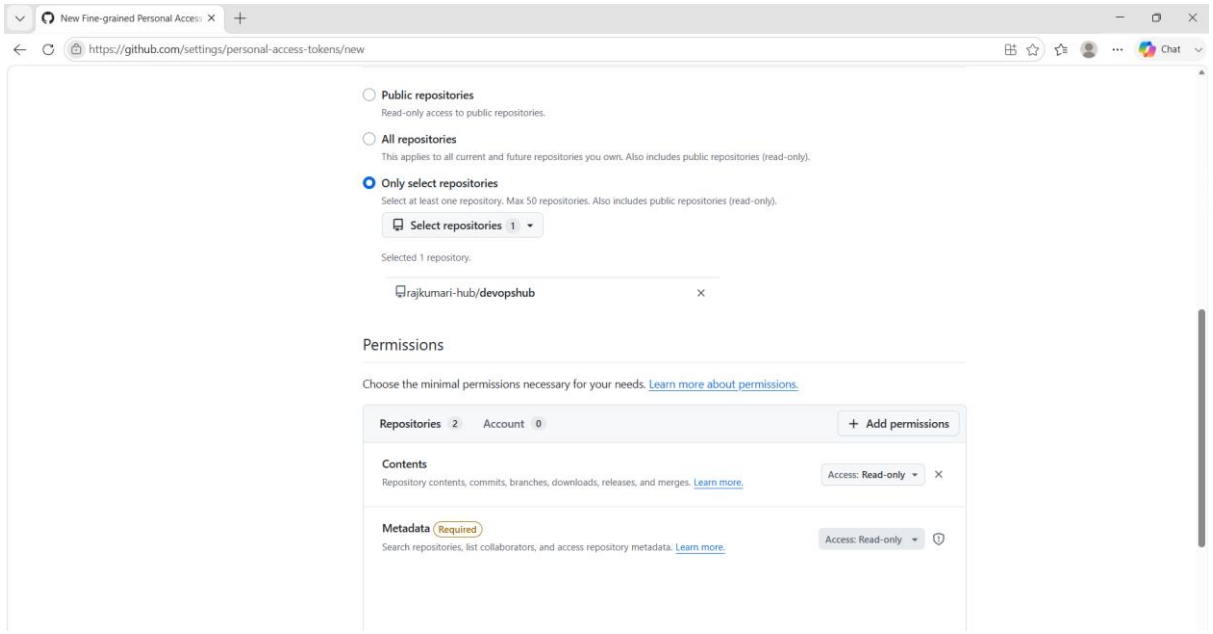
Create a token with permission to access your private repository.

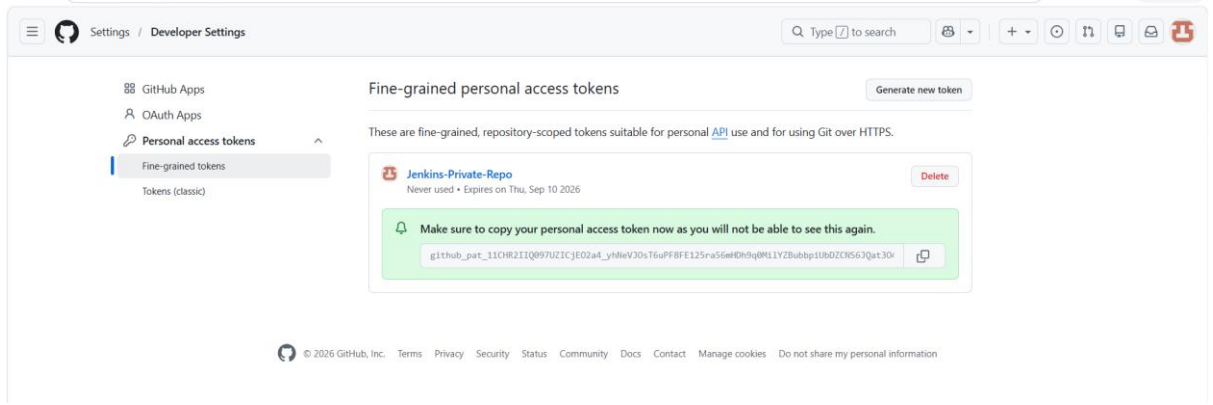
For a fine-grained token, give it access to the required private repository and enable appropriate repository permissions, especially **Contents: Read**.

Do not share the token with anyone.









3. Add GitHub Credentials to Jenkins

Go to:

Jenkins → Manage Jenkins → Credentials

Then:

System → Global credentials → Add Credentials

Select:

Kind: Username with password

Enter:

Username: <your GitHub username>

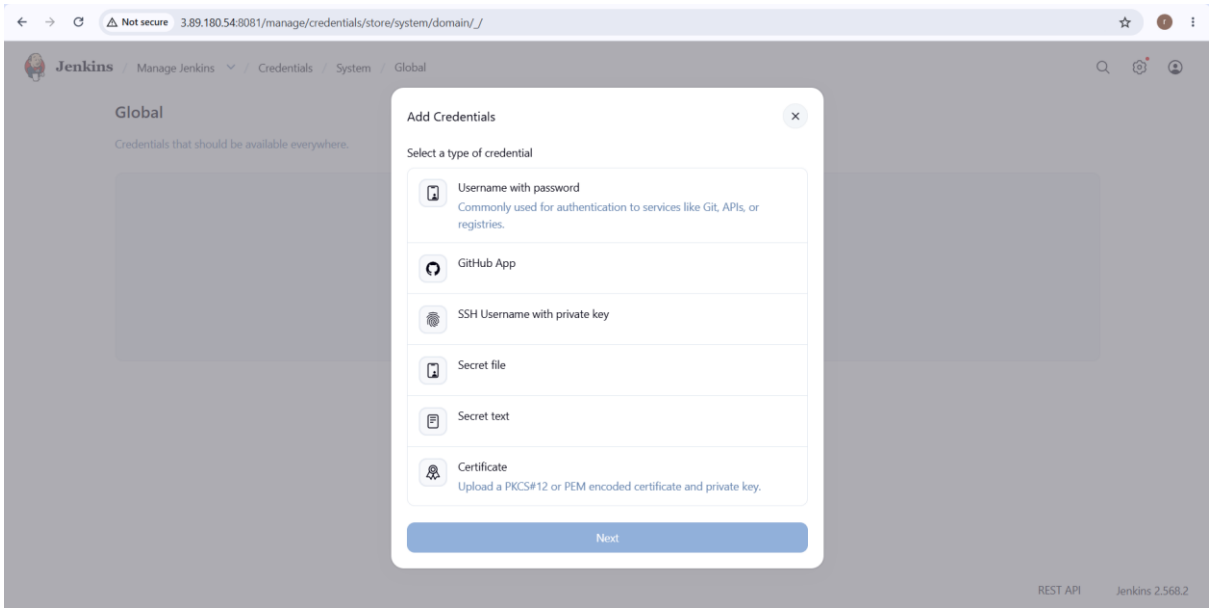
Password: <your GitHub Personal Access Token>

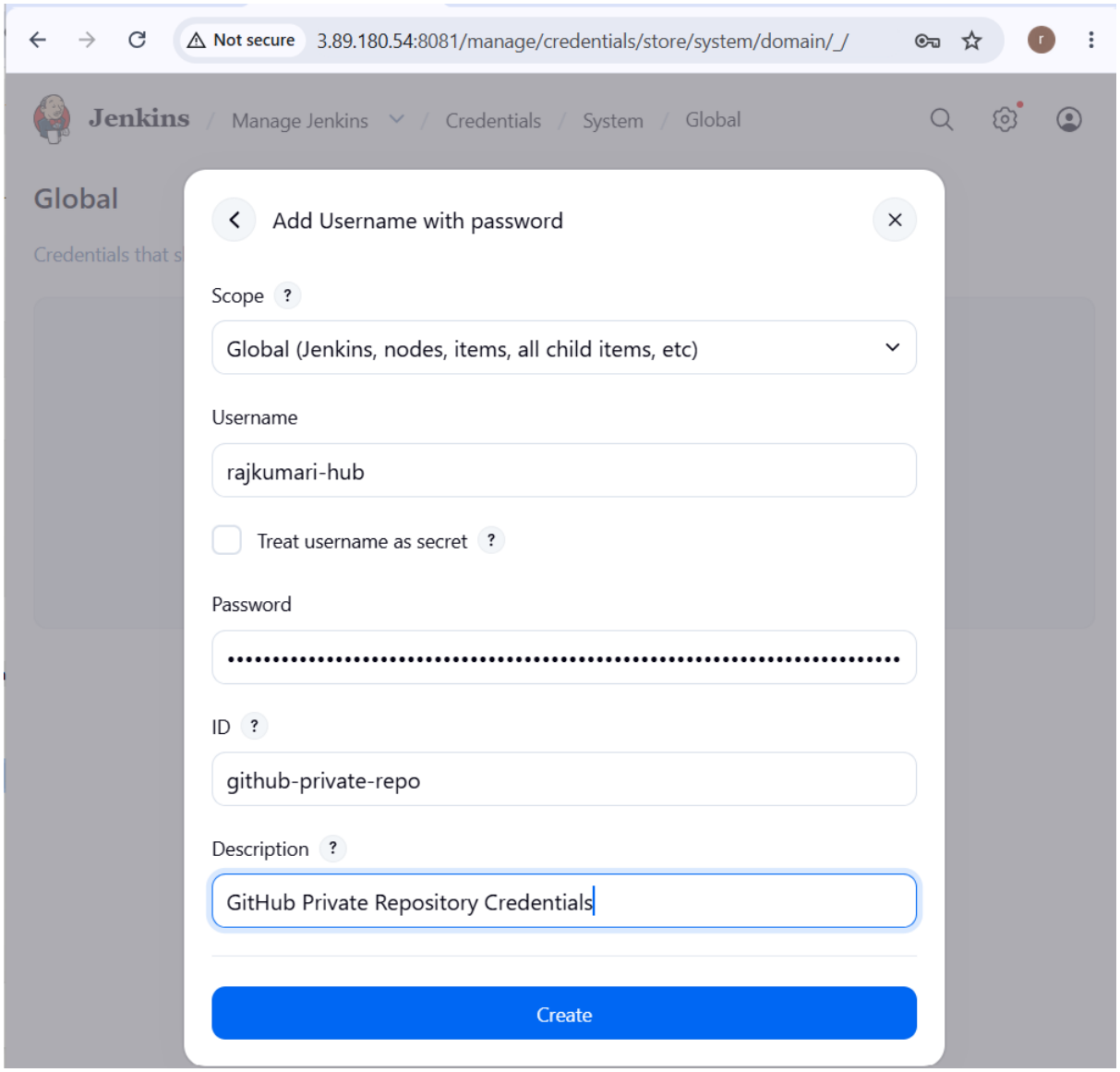
ID: github-private-repo

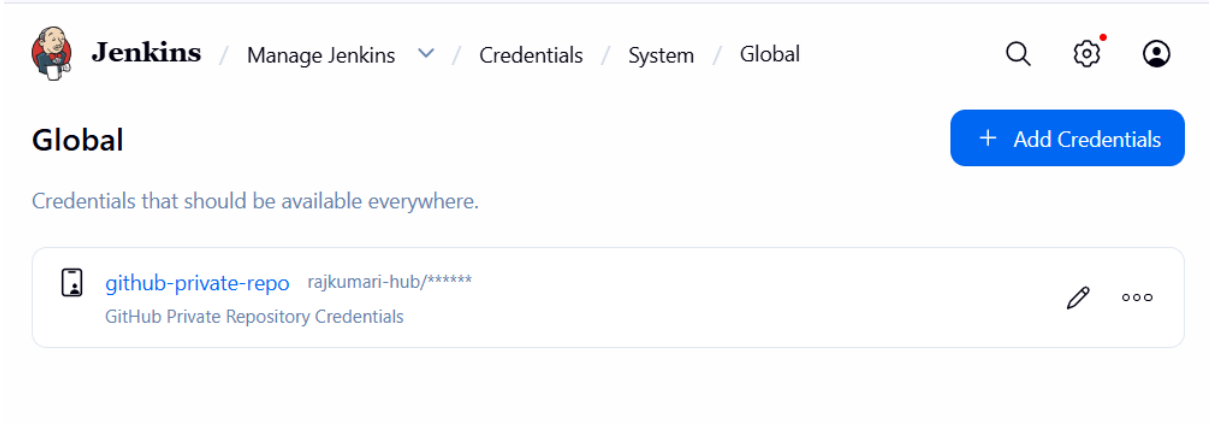
Description: GitHub Private Repository Credentials

Click:

Create







4. Create the Jenkins Job

Go to:

Jenkins Dashboard → New Item

Enter:

DevOps-Private-GitHub-Build

Select:

Freestyle project

Click **OK**.

5. Configure the Agent

Under **General**, select:

Restrict where this project can be run

Enter:

devops-node

This makes the job run on your Devops-agent.

6. Configure GitHub Private Repository

Go to:

Source Code Management → Git

Enter your private repository URL:

https://github.com/<your-username>/Jenkins-Private-App.git

Under **Credentials**, select:

github-private-repo

For **Branch Specifier**, enter:

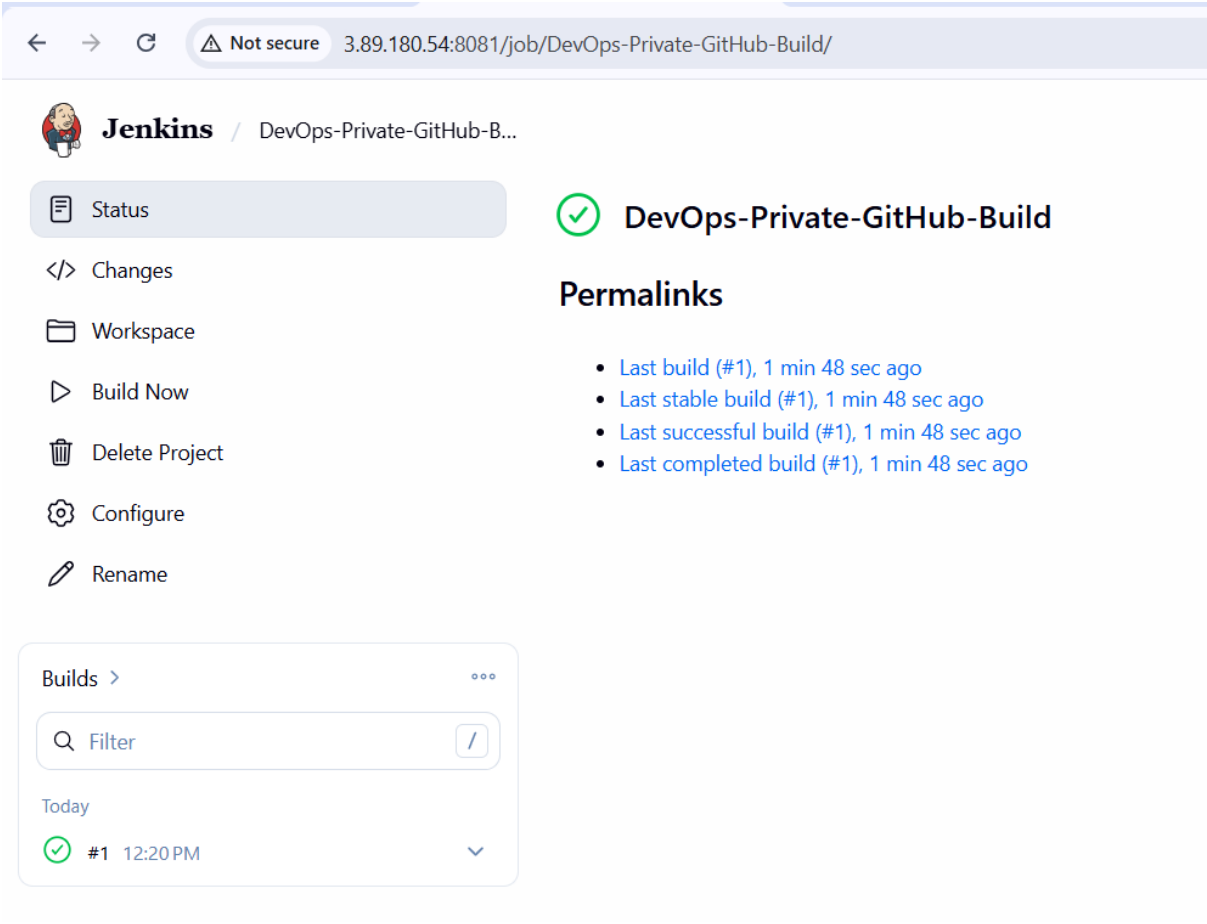
*/main

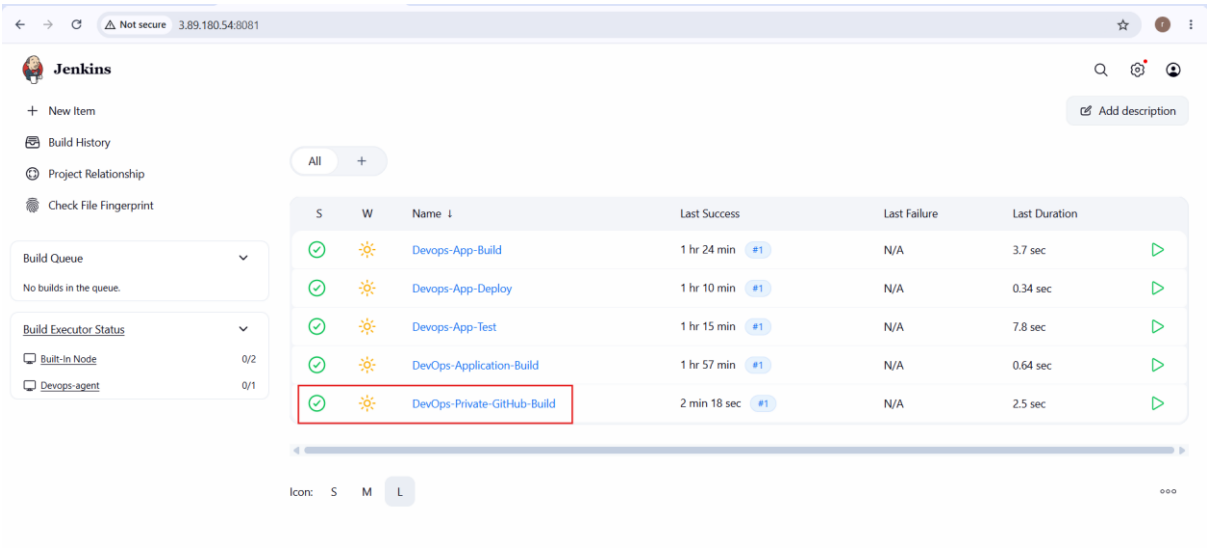
Jenkins will now authenticate with GitHub before cloning the private repository.

7. Add a Build Step

Go to:

Build Steps → Add build step → Execute shell





Final flow

GitHub Private Repository

|

| HTTPS + PAT

▼

Jenkins

|

| Label: devops-node

▼

Devops-agent

|

▼

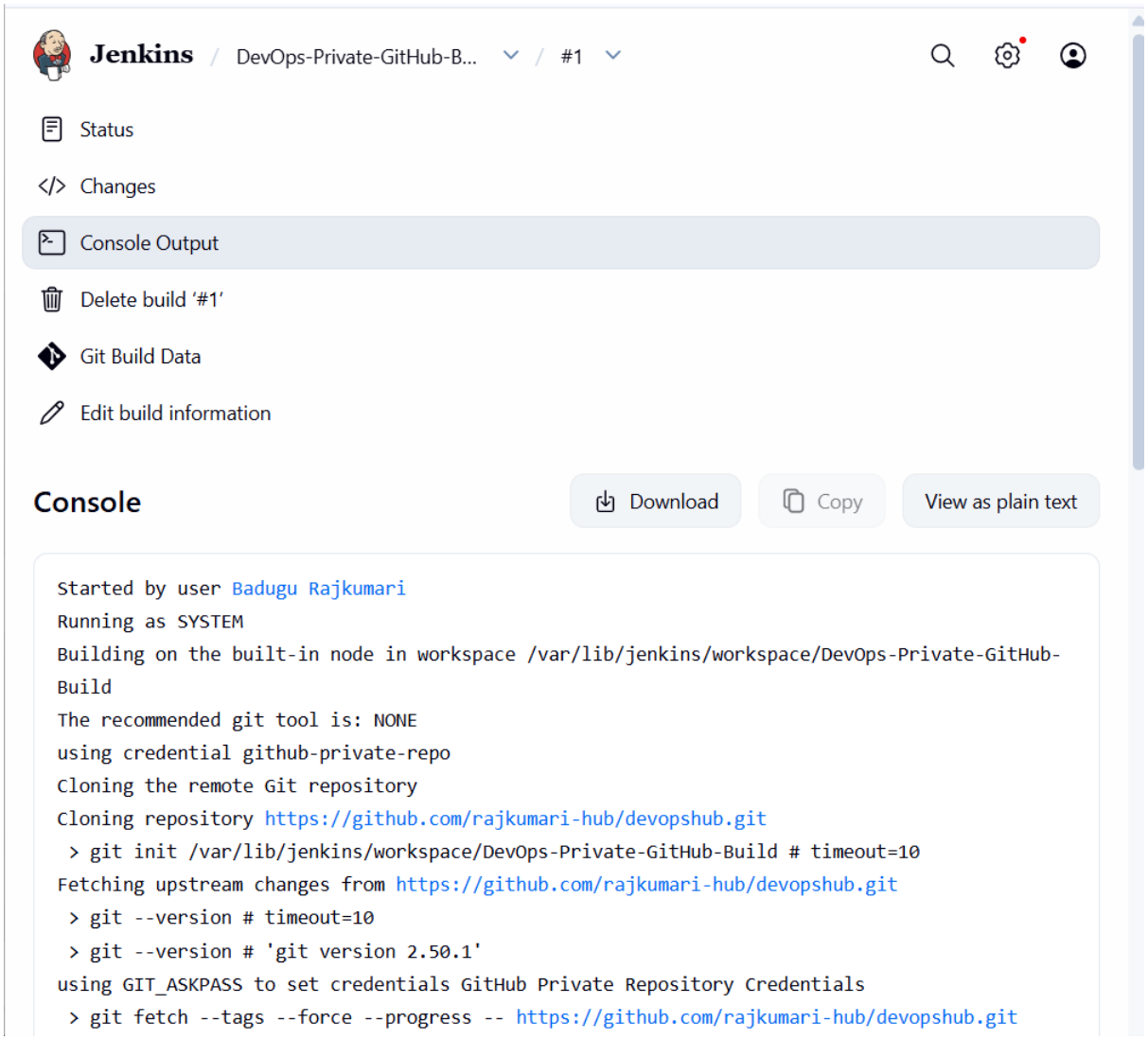
Build Job

|

▼

SUCCESS

Documentation result: Jenkins successfully authenticates with a private GitHub repository using stored Jenkins credentials, checks out the source code onto the Devops-agent, and executes the configured build step.


Jenkins / DevOps-Private-GitHub-B... / #1
Status
Changes
Console Output
Delete build '#1'
Git Build Data
Edit build information
Console
Download
Copy
View as plain text
Started by user Badugu Rajkumari
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/DevOps-Private-GitHub-Build
The recommended git tool is: NONE
using credential github-private-repo
Cloning the remote Git repository
Cloning repository https://github.com/rajkumari-hub/devopshub.git
> git init /var/lib/jenkins/workspace/DevOps-Private-GitHub-Build # timeout=10
Fetching upstream changes from https://github.com/rajkumari-hub/devopshub.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials GitHub Private Repository Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumari-hub/devopshub.git